

Module X2

Extensions PostgreSQL pour la performance



25.09

Table des matières

Sur ce document	1
Chers lectrices & lecteurs,	1
À propos de DALIBO	1
Remerciements	2
Forme de ce manuel	2
Licence Creative Commons CC-BY-NC-SA	2
Marques déposées	3
Versions de PostgreSQL couvertes	3
1/ Extensions PostgreSQL pour la performance	5
1.1 Préambule	6
1.2 pg_trgm	7
1.3 pg_stat_statements	9
1.3.1 pg_stat_statements : mise en place	10
1.3.2 pg_stat_statements : exemple 1	11
1.3.3 pg_stat_statements : exemple 2	12
1.3.4 pg_stat_statements : exemple 3	14
1.4 auto_explain	15
1.5 pg_buffercache	20
1.6 pg_prewarm	25
1.7 Langages procéduraux	27
1.7.1 Avantages & inconvénients	28
1.8 hll	30
1.9 Quiz	32
1.10 Travaux pratiques	33
1.10.1 Indexation de pattern avec les varchar_patterns et pg_trgm	33
1.10.2 auto_explain	34
1.10.3 pg_stat_statements	34
1.10.4 PL/Python, import de page web et compression	35
1.10.5 PL/Perl et comparaison de performances	36
1.10.6 hll	38
1.11 Travaux pratiques (solutions)	40
1.11.1 Indexation de pattern avec les varchar_patterns et pg_trgm	40
1.11.2 auto_explain	44
1.11.3 pg_stat_statements	45
1.11.4 PL/Python, import de page web et compression	50
1.11.5 PL/Perl et comparaison de performances	53
1.11.6 hll	57
Les formations Dalibo	63
Cursus des formations	63
Les livres blancs	64

Téléchargement gratuit	64
----------------------------------	----

Sur ce document

Formation	Module X2
Titre	Extensions PostgreSQL pour la performance
Révision	25.09
PDF	https://dali.bo/x2_pdf
EPUB	https://dali.bo/x2_epub
HTML	https://dali.bo/x2_html
Slides	https://dali.bo/x2_slides
TP	https://dali.bo/x2_tp
TP (solutions)	https://dali.bo/x2_solutions

Vous trouverez en ligne les différentes versions complètes de ce document.

Chers lectrices & lecteurs,

Nos formations PostgreSQL sont issues de nombreuses années d'études, d'expérience de terrain et de passion pour les logiciels libres. Pour Dalibo, l'utilisation de PostgreSQL n'est pas une marque d'opportunisme commercial, mais l'expression d'un engagement de longue date. Le choix de l'Open Source est aussi le choix de l'implication dans la communauté du logiciel.

Au-delà du contenu technique en lui-même, notre intention est de transmettre les valeurs qui animent et unissent les développeurs de PostgreSQL depuis toujours : partage, ouverture, transparence, créativité, dynamisme... Le but premier de nos formations est de vous aider à mieux exploiter toute la puissance de PostgreSQL mais nous espérons également qu'elles vous inciteront à devenir un membre actif de la communauté en partageant à votre tour le savoir-faire que vous aurez acquis avec nous.

Nous mettons un point d'honneur à maintenir nos manuels à jour, avec des informations précises et des exemples détaillés. Toutefois malgré nos efforts et nos multiples relectures, il est probable que ce document contienne des oublis, des coquilles, des imprécisions ou des erreurs. Si vous constatez un souci, n'hésitez pas à le signaler via l'adresse formation@dalibo.com¹ !

À propos de DALIBO

DALIBO est le spécialiste français de PostgreSQL. Nous proposons du support, de la formation et du conseil depuis 2005.

Retrouvez toutes nos formations sur <https://dalibo.com/formations>

¹<mailto:formation@dalibo.com>

Remerciements

Ce manuel de formation est une aventure collective qui se transmet au sein de notre société depuis des années. Nous remercions chaleureusement ici toutes les personnes qui ont contribué directement ou indirectement à cet ouvrage, notamment :

Alexandre Anriot, Jean-Paul Argudo, Carole Arnaud, Alexandre Baron, David Bidoc, Sharon Bonan, Franck Boudehen, Arnaud Bruniquel, Pierrick Chovelon, Damien Clochard, Christophe Courtois, Marc Cousin, Gilles Darold, Ronan Dunklau, Vik Fearing, Stefan Fercot, Dimitri Fontaine, Pierre Giraud, Nicolas Gollet, Nizar Hamadi, Florent Jardin, Virginie Jourdan, Luc Lamarle, Denis Laxalde, Guillaume Lelarge, Alain Lesage, Benoit Lobréau, Jean-Louis Louër, Thibaut Madelaine, Cédric Martin, Adrien Nayrat, Alexandre Pereira, Flavie Perette, Robin Portigliatti, Thomas Reiss, Maël Rimbault, Jehan-Guillaume de Rorthais, Julien Rouhaud, Stéphane Schildknecht, Julien Tachaires, Nicolas Thauvin, Be Hai Tran, Christophe Truffier, Arnaud de Vathaire, Cédric Villemain, Thibaud Walkowiak, Frédéric Yhuel.

Forme de ce manuel

Les versions PDF, EPUB ou HTML de ce document sont structurées autour des slides de nos formations. Le texte suivant chaque slide contient le cours et de nombreux détails qui ne peuvent être données à l'oral.

Licence Creative Commons CC-BY-NC-SA

Cette formation est sous licence **CC-BY-NC-SA**². Vous êtes libre de la redistribuer et/ou modifier aux conditions suivantes :

- Paternité
- Pas d'utilisation commerciale
- Partage des conditions initiales à l'identique

Vous n'avez pas le droit d'utiliser cette création à des fins commerciales.

Si vous modifiez, transformez ou adaptez cette création, vous n'avez le droit de distribuer la création qui en résulte que sous un contrat identique à celui-ci.

Vous devez citer le nom de l'auteur original de la manière indiquée par l'auteur de l'œuvre ou le titulaire des droits qui vous confère cette autorisation (mais pas d'une manière qui suggérerait qu'ils vous soutiennent ou approuvent votre utilisation de l'œuvre). À chaque réutilisation ou distribution de cette création, vous devez faire apparaître clairement au public les conditions contractuelles de sa mise à disposition. La meilleure manière de les indiquer est un lien vers cette page web. Chacune de ces conditions peut être levée si vous obtenez l'autorisation du titulaire des droits sur cette œuvre. Rien dans ce contrat ne diminue ou ne restreint le droit moral de l'auteur ou des auteurs.

Le texte complet de la licence est disponible sur <http://creativecommons.org/licenses/by-nc-sa/2.0/fr/legalcode>

²<http://creativecommons.org/licenses/by-nc-sa/2.0/fr/legalcode>

Cette licence interdit la réutilisation pour l'apprentissage d'une IA. Elle couvre les diapositives, les manuels eux-mêmes et les travaux pratiques.

Cette formation peut également contenir quelques images et schémas dont la redistribution est soumise à des licences différentes qui sont alors précisées.

Marques déposées

PostgreSQL® Postgres® et le logo Slonik sont des marques déposées³ par PostgreSQL Community Association of Canada.

Versions de PostgreSQL couvertes

Ce document ne couvre que les versions supportées de PostgreSQL au moment de sa rédaction, soit les versions 13 à 17.

Sur les versions précédentes susceptibles d'être encore rencontrées en production, seuls quelques points très importants sont évoqués, en plus éventuellement de quelques éléments historiques.

Sauf précision contraire, le système d'exploitation utilisé est Linux.

³<https://www.postgresql.org/about/policies/trademarks/>

1/ Extensions PostgreSQL pour la performance



1.1 PRÉAMBULE



Ce module présente des extensions plus spécifiquement destinées à améliorer les performances.

1.2 PG_TRGM



- Indexation des recherches `LIKE '%critère%'`
- Similarité basée sur des trigrammes

```
CREATE EXTENSION pg_trgm;
SELECT similarity('bonjour', 'bnojour');
```

```
similarity
```

```
-----
0.333333
```

- Indexation (GIN ou GiST) :

```
CREATE INDEX test_trgm_idx ON test_trgm
USING gist (text_data gist_trgm_ops);
```

Ce module permet de décomposer en trigramme les chaînes qui lui sont proposées :

```
SELECT show_trgm('hello');
```

```
show_trgm
```

```
-----
{" h"," he","ell","hel","llo","lo "}
```

Une fois les trigrammes indexés, on peut réaliser de la recherche floue, ou utiliser des clauses `LIKE` malgré la présence de jokers (`%`) n'importe où dans la chaîne. À l'inverse, les indexations simples, de type B-tree, ne permettent des recherches efficaces que dans un cas particulier : si le seul joker de la chaîne est à la fin de celle-ci (`LIKE 'hello%'` par exemple). Contrairement à la *Full Text Search*, la recherche par trigrammes ne réclame aucune modification des requêtes.

```
CREATE EXTENSION pg_trgm;
```

```
CREATE TABLE test_trgm (text_data text);
```

```
INSERT INTO test_trgm(text_data)
VALUES ('hello'), ('hello everybody'),
('he lo young man'), ('hallo!'), ('HELLO !');
INSERT INTO test_trgm SELECT 'hola' FROM generate_series(1,1000);
```

```
CREATE INDEX test_trgm_idx ON test_trgm
USING gist (text_data gist_trgm_ops);
```

```
SELECT text_data FROM test_trgm
WHERE text_data like '%hello%';
```

```
text_data
```

```
-----
hello
hello everybody
```

Cette dernière requête passe par l'index `test_trgm_idx`, malgré le `%` initial :

```
EXPLAIN (ANALYZE)
SELECT text_data FROM test_trgm
WHERE text_data like '%hello%' ;
```

QUERY PLAN

```
-----
Index Scan using test_trgm_gist_idx on test_trgm
  (cost=0.41..0.63 rows=1 width=8) (actual time=0.174..0.204 rows=2 loops=1)
    Index Cond: (text_data ~~ '%hello% '::text)
    Rows Removed by Index Recheck: 1
    Planning time: 0.202 ms
    Execution time: 0.250 ms
```

On peut aussi utiliser un index GIN (comme pour le *Full Text Search*). Les index GIN ont l'avantage d'être plus efficaces pour les recherches exhaustives. Mais l'indexation pour la recherche des k éléments les plus proches (on parle de recherche k-NN) n'est disponible qu'avec les index GiST .

```
SELECT text_data, text_data <-> 'hello'
FROM test_trgm
ORDER BY text_data <-> 'hello'
LIMIT 4;
```

nous retourne par exemple les deux enregistrements les plus proches de « hello » dans la table `test_trgm`.

1.3 PG_STAT_STATEMENTS



Capture en temps réel des requêtes

- Normalisation
- Indicateurs :
 - nombre d'exécutions,
 - nombre d'enregistrements retournés
 - temps cumulé d'exécution et d'optimisation
 - lectures/écritures en cache, demandées au système, tris
 - temps de lecture/écriture (`track_io_timing`)
 - écritures dans les journaux de transactions
 - temps de planning (désactivé par défaut)
 - utilisation du JIT (v15)

Cette extension est fournie avec PostgreSQL et est parmi les plus populaires et les plus utiles.

Une fois installé, `pg_stat_statements` capture, à chaque exécution de requête, tous les compteurs ci-dessus et d'autres associés à cette requête (champ `query`), ci-dessous avec PostgreSQL 17 :

```
postgres=# \d pg_stat_statements
```

Vue « public.pg_stat_statements »				
Colonne	Type	Collationnement	NULL-able	Par défaut
userid	oid			
dbid	oid			
toplevel	boolean			
queryid	bigint			
query	text			
plans	bigint			
total_plan_time	double precision			
min_plan_time	double precision			
max_plan_time	double precision			
mean_plan_time	double precision			
stddev_plan_time	double precision			
calls	bigint			
total_exec_time	double precision			
min_exec_time	double precision			
max_exec_time	double precision			
mean_exec_time	double precision			
stddev_exec_time	double precision			
rows	bigint			
shared_blks_hit	bigint			
shared_blks_read	bigint			
shared_blks_dirtied	bigint			
shared_blks_written	bigint			
local_blks_hit	bigint			
local_blks_read	bigint			
local_blks_dirtied	bigint			
local_blks_written	bigint			
temp_blks_read	bigint			
temp_blks_written	bigint			
shared_blk_read_time	double precision			
shared_blk_write_time	double precision			
local_blk_read_time	double precision			

local_blk_write_time	double precision			
temp_blk_read_time	double precision			
temp_blk_write_time	double precision			
wal_records	bigint			
wal_fpi	bigint			
wal_bytes	numeric			
jit_functions	bigint			
jit_generation_time	double precision			
jit_inlining_count	bigint			
jit_inlining_time	double precision			
jit_optimization_count	bigint			
jit_optimization_time	double precision			
jit_emission_count	bigint			
jit_emission_time	double precision			
jit_deform_count	bigint			
jit_deform_time	double precision			
stats_since	timestamp with time zone			
minmax_stats_since	timestamp with time zone			

Quelques champs peuvent manquer ou porter un autre nom dans les versions précédentes.

Les requêtes d'une même base et d'un même utilisateur sont normalisées (reconnues comme identiques même avec des paramètres différents).

Les champs sont détaillés dans https://dali.bo/h2_html#pg_stat_statements.

1.3.1 pg_stat_statements : mise en place



```
shared_preload_libraries = 'pg_stat_statements'
```

```
CREATE EXTENSION pg_stat_statements ; -- dans 1 ou plusieurs bases
SELECT * FROM pg_stat_statements ;
```

- Vue en mémoire partagée (volumétrie contrôlée)
- Pas d'échantillonnage, seulement des compteurs cumulés
- `pg_stat_statements_reset()` ou PoWA

1.3.1.1 Installation et réinitialisation

Ce module nécessite un espace en mémoire partagée. Pour l'installer, il faut donc renseigner le paramètre suivant avant de redémarrer l'instance :

```
shared_preload_libraries = 'pg_stat_statements'
```

Il faut installer l'extension dans au moins une base (dont une à laquelle les développeurs auront aussi accès, car l'information les concerne au premier chef) :

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements ;
```

La vue `pg_stat_statements` retourne un instantané des compteurs au moment de l'interrogation depuis l'installation, depuis le dernier arrêt brutal, ou depuis le dernier appel à la fonction `pg_stat_statements_reset()`. Cette dernière fonction permet de réinitialiser les compteurs pour une base, un utilisateur, une requête, ou tout.

Deux méthodes d'utilisation sont donc possibles :

- effectuer un *reset* au début d'une période, puis interroger la vue `pg_stat_statements` à la fin de cette période;
- capturer à intervalles réguliers le contenu de `pg_stat_statements` et visualiser les changements dans les compteurs : le projet PoWA¹ a été développé à cet effet.

La requête étant déjà analysée, cette opération supplémentaire n'ajoute qu'un faible surcoût (de l'ordre de 5 % sur une requête extrêmement courte), fixe, pour chaque requête.

Les données de l'extension sont stockées dans le PGDATA, sous `pg_stat_tmp` (même pour les versions récentes de PostgreSQL qui ne l'utilisent plus pour le `stats collector`), et un arrêt brutal peut mener à la perte du contenu.

1.3.1.2 Paramétrage

`pg_stat_statements` possède quelques paramètres².

Dès lors que l'extension est chargée en mémoire, la capture des compteurs est enclenchée, sauf si le paramètre `pg_stat_statements.track` est positionné à `none`. Celui-ci permet donc d'activer cette capture à la demande, sans qu'il soit nécessaire de redémarrer l'instance, ce qui peut s'avérer utile pour une instance avec beaucoup de requêtes très courtes (de type OLTP), et dont la rapidité est un élément critique : pour une telle instance, le surcoût lié à `pg_stat_statements` peut être jugé trop important pour que cette capture soit activée en permanence.

Sur un serveur chargé, il est déconseillé de réduire `pg_stat_statements.max` (nombre de requêtes différentes suivies, à 5000 par défaut), car le coût d'une désallocation n'est pas négligeable³.

1.3.2 pg_stat_statements : exemple 1



Requêtes les plus longues en temps cumulé :

```
SELECT r.rolname, d.datname, s.calls, s.total_exec_time,
       s.total_exec_time / s.calls AS avg_time, s.query
FROM   pg_stat_statements s
JOIN   pg_roles r         ON (s.userid=r.oid)
JOIN   pg_database d      ON (s.dbid = d.oid)
ORDER BY s.total_exec_time DESC
LIMIT 10 ;
```

¹<https://powa.readthedocs.io/en/latest/>

²<https://docs.postgresql.fr/current/pgstatstatements.html#id-1.11.7.40.9>

³https://yhuelf.github.io/2021/09/30/pg_stat_statements_bottleneck.html

La requête ci-dessus affiche les dix requêtes les plus longues en cumulé (même avec des paramètres différents), le nombre d'appels, le temps total, le temps moyen par appel. Les temps sont en millisecondes.

NB : pour une instance en version 12 ou antérieure, utiliser le champ `total_time`, qui inclut aussi le temps de planification.

1.3.3 pg_stat_statements : exemple 2



Requêtes les plus fréquemment appelées :

```
SELECT r.rolname, d.datname, s.calls, s.total_exec_time,
       s.total_exec_time / s.calls AS avg_time, s.query
FROM   pg_stat_statements s
JOIN   pg_roles r         ON (s.userid=r.oid)
JOIN   pg_database d      ON (s.dbid = d.oid)
ORDER BY s.calls DESC
LIMIT 10;
```

Cette requête affiche les dix requêtes les plus fréquentes en nombre d'appels, et le temps moyen. Exemple de sortie, avec un peu de formatage :

```
\pset format wrapped
\pset columns 83
```

```
SELECT r.rolname, d.datname,
       to_char(s.calls, '999G999FM') AS calls,
       s.total_exec_time * interval '1ms' AS total_exec_time,
       s.total_exec_time/s.calls * interval '1ms' AS avg_time,
       s.query
FROM   pg_stat_statements s
JOIN   pg_roles r         ON (s.userid=r.oid)
JOIN   pg_database d      ON (s.dbid = d.oid)
ORDER BY s.calls DESC
LIMIT 10 \gx
```

```
-[ RECORD 1 ]-----+-----
rolname      | postgres
datname      | postgres
calls        | 329 021
total_exec_time | 00:00:01.617168
avg_time     | 00:00:00.000005
query        | SELECT pg_postmaster_start_time()
-[ RECORD 2 ]-----+-----
rolname      | postgres
datname      | postgres
calls        | 316 192
total_exec_time | 24:19:01.780477
avg_time     | 00:00:00.276863
query        | SELECT                                     +
                                     count(datid) as databases,             +
                                     pg_size_pretty(sum(pg_database_size(
```

DALIBO Formations

	pg_database.datname)::bigint) as total_size,	+
	to_char(now(),\$1) as time,	+
	sum(xact_commit)::BIGINT as total_commit,	+
	sum(xact_rollback)::BIGINT as total_rollback	+
	FROM pg_database	+
	JOIN pg_stat_database ON (pg_database.oid = pg_stat_data.	+
	base.datid)	+
	WHERE datistemplate = \$2	
-[RECORD 3]----		
rolname	postgres	
datname	postgres	
calls	316 192	
total_exec_time	00:01:22.127931	
avg_time	00:00:00.00026	
query	SELECT CASE sum(blks_hit+blks_read)	+
	WHEN \$1 THEN \$2	+
	ELSE trunc(sum(blks_hit)/sum(blks_hit+blks_read)*\$3)::	+
	.float	+
	END AS hitratio	+
	FROM pg_stat_database	
-[RECORD 4]----		
rolname	postgres	
datname	postgres	
calls	316 192	
total_exec_time	00:00:02.82872	
avg_time	00:00:00.000009	
query	SELECT buffers_alloc FROM pg_stat_bgwriter	
-[RECORD 5]----		
rolname	postgres	
datname	postgres	
calls	316 192	
total_exec_time	00:18:08.125136	
avg_time	00:00:00.003441	
query	SELECT COUNT(*) AS nb FROM pg_stat_activity WHERE state != \$1	
-[RECORD 6]----		
rolname	postgres	
datname	pgbench_300_hdd	
calls	79 534	
total_exec_time	00:03:44.82423	
avg_time	00:00:00.002827	
query	select wait_event, wait_event_type, query from pg_stat_activity .	
	.where state =\$1 and pid = \$2	
-[RECORD 7]----		
rolname	temboard_agent	
datname	postgres	
calls	75 028	
total_exec_time	00:00:00.368735	
avg_time	00:00:00.000005	
query	SELECT pg_postmaster_start_time()	
-[RECORD 8]----		
rolname	temboard_agent	
datname	postgres	
calls	72 091	
total_exec_time	00:04:02.992142	
avg_time	00:00:00.003371	
query	SELECT COUNT(*) AS nb FROM pg_stat_activity WHERE state != \$1	
-[RECORD 9]----		

```

rolname      | temboard_agent
datname      | postgres
calls       | 72 091
total_exec_time | 05:47:55.416569
avg_time     | 00:00:00.28957
query       | SELECT
                                     count(datid) as databases,
                                     pg_size_pretty(sum(pg_database_size(
                                     pg_database.datname))::bigint) as total_size,
                                     to_char(now(),$1) as time,
                                     sum(xact_commit)::BIGINT as total_commit,
                                     sum(xact_rollback)::BIGINT as total_rollback
                                     FROM pg_database
                                     JOIN pg_stat_database ON (pg_database.oid = pg_stat_data.
                                     .base.datid)
                                     WHERE datistemplate = $2
-[ RECORD 10 ]--
rolname      | temboard_agent
datname      | postgres
calls       | 72 091
total_exec_time | 00:00:17.817369
avg_time     | 00:00:00.000247
query       | SELECT CASE sum(blks_hit+blks_read)
                                     WHEN $1 THEN $2
                                     ELSE trunc(sum(blks_hit)/sum(blks_hit+blks_read)*$3)::
                                     .float
                                     END AS hitratio
                                     FROM pg_stat_database

```

On voit qu'il y a beaucoup de requêtes de supervision, ce qui est logique. Il est donc conseillé de dédier un utilisateur à la supervision pour pouvoir filtrer aisément.

1.3.4 pg_stat_statements : exemple 3



Requêtes les plus consommatrices et *hit ratio* :

```

SELECT calls, total_exec_time, rows,
       100.0*shared_blks_hit
       /nullif(shared_blks_hit+shared_blks_read, 0) AS "hit %",
       query
FROM   pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 5 ;

```

Cette requête calcule le *hit ratio*, c'est-à-dire la proportion des blocs lus depuis le cache de PostgreSQL, pour les cinq plus grosses requêtes en temps cumulé. Dans l'idéal, ce ratio serait à 100 %.

1.4 AUTO_EXPLAIN



- Tracer les plans des requêtes lentes automatiquement
- Contrib officielle
- Mise en place globale (traces) :
 - globale :


```
shared_preload_libraries='auto_explain'  -- redémarrage !
ALTER DATABASE erp SET auto_explain.log_min_duration = '3s' ;
```
 - session :


```
LOAD 'auto_explain' ;
SET auto_explain.log_analyze TO true;
```

Principe :

L'outil `auto_explain` est habituellement activé quand on a le sentiment qu'une requête devient subitement lente à certains moments, et qu'on suspecte que son plan diffère entre deux exécutions. Elle permet de tracer dans les journaux applicatifs, voire dans la console, le plan de la requête dès qu'elle dépasse une durée configurée.

C'est une « contrib » officielle de PostgreSQL (et non une extension). Tracer systématiquement le plan d'exécution d'une requête fréquente prend de la place, et est assez coûteux. C'est donc un outil à utiliser parcimonieusement, qui par défaut ne trace rien (paramètre `auto_explain.log_min_duration` à -1).

Activation et utilisation dans une session :

Pour utiliser `auto_explain` dans la session en cours, il faut activer quelques options similaires à celles d'`EXPLAIN` :

```
-- Chargement du module
LOAD 'auto_explain' ;
-- Durée minimale pour tracer (ms), le défaut est -1
SET auto_explain.log_min_duration = '0';
-- Options à activer
SET auto_explain.log_analyze TO on ;
SET auto_explain.log_buffers TO on ;
SET auto_explain.log_verbose TO on ;
```

Les plans se retrouvent dans les traces de l'instance. Pour les afficher aussi dans la session en cours, il faut descendre le niveau des messages affichés à `LOG` (le défaut de `auto_explain`) :

```
SET client_min_messages to log;
```

Ensuite, lancer la requête à analyser : le plan apparaît suivi du résultat de la requête.

```
SELECT count(*)
FROM pg_class, pg_index
WHERE oid = indrelid AND indisunique;
```

```

LOG:  duration: 0.652 ms  plan:
Query Text: SELECT count(*)
           FROM pg_class, pg_index
           WHERE oid = indrelid AND indisunique;
Aggregate  (cost=29.75..29.76 rows=1 width=8) (actual time=0.630..0.632 rows=1 loops=1)
Output: count(*)
Buffers: shared hit=18
-> Hash Join  (cost=23.34..29.37 rows=150 width=0) (actual time=0.532..0.612 rows=156 loops=1)
    Inner Unique: true
    Hash Cond: (pg_index.indrelid = pg_class.oid)
    Buffers: shared hit=18
    -> Seq Scan on pg_catalog.pg_index  (cost=0.00..5.64 rows=150 width=4) (actual time=0.014..0.051 rows=156 loops=1)
        Output: pg_index.indexrelid, pg_index.indrelid, pg_index.indnatts, pg_index.indkeyatts, pg_index.indisunique, pg_index.indnullsnotdistinct
        Filter: pg_index.indisunique
        Rows Removed by Filter: 14
        Buffers: shared hit=4
    -> Hash  (cost=18.15..18.15 rows=415 width=4) (actual time=0.310..0.311 rows=436 loops=1)
        Output: pg_class.oid
        Buckets: 1024  Batches: 1  Memory Usage: 24kB
        Buffers: shared hit=14
    -> Seq Scan on pg_catalog.pg_class  (cost=0.00..18.15 rows=415 width=4) (actual time=0.018..0.191 rows=436 loops=1)
        Output: pg_class.oid
        Buffers: shared hit=14
Query Identifier: 7458316119861708727
LOG:  duration: 3.398 ms  statement: SELECT count(*)
           FROM pg_class, pg_index
           WHERE oid = indrelid AND indisunique;
count
-----
    156
(1 row)

```

Autres paramètres :

L'outil a d'autres options, qui reprennent souvent les paramètres habituels d'`EXPLAIN`. Leurs valeurs par défaut sont les suivantes :

SELECT **name**, setting **FROM** pg_settings **WHERE** **name** **LIKE** 'auto_explain%';

name	setting
auto_explain.log_analyze	on
auto_explain.log_buffers	off
auto_explain.log_format	text
auto_explain.log_level	log
auto_explain.log_min_duration	0
auto_explain.log_nested_statements	off
auto_explain.log_parameter_max_length	-1
auto_explain.log_settings	off
auto_explain.log_timing	on
auto_explain.log_triggers	off
auto_explain.log_verbose	off
auto_explain.log_wal	off
auto_explain.sample_rate	1

Voir la documentation⁴ pour ce qui n'est pas traité ici.

Utilisation au niveau global et précautions :

Pour utiliser `auto_explain` globalement, il faut charger la bibliothèque au démarrage dans le fichier `postgresql.conf` via le paramètre `shared_preload_libraries`.

`shared_preload_libraries='auto_explain'`

⁴<https://docs.postgresql.fr/current/auto-explain.html#AUTO-EXPLAIN-CONFIGURATION-PARAMETERS>

Après un redémarrage de l'instance, il est possible de configurer les paramètres de capture des plans d'exécution par base de données. Dans l'exemple ci-dessous, l'ensemble des requêtes sont tracées sur la base de données `bench`, qui est utilisée par `pgbench`.

```
ALTER DATABASE bench SET auto_explain.log_min_duration = '0';
ALTER DATABASE bench SET auto_explain.log_analyze = on ;
```



Attention! L'activation des traces complètes sur une base de données avec un fort volume de requêtes, et/ou des requêtes aux plans complexes, peut faire gonfler énormément les traces (`postgresql.log`). N'activez donc `auto_explain` que de manière limitée.



Pour ne tracer qu'un échantillonnage des requêtes, déclarer par exemple :

```
ALTER DATABASE bench SET auto_explain.sample_rate = 0.01 ;
```



Un autre problème, rare mais déjà rencontré, est l'activation simultanée du paramètre `track_io_timing`, alors que le test avec `pg_test_timing` indique que c'est problématique. L'impact sur les performances peut être sévère.

`pg_test_timing`⁵ est livré avec PostgreSQL et mesure les performances de l'horloge système. Si le temps de mesure renvoyé sur la deuxième ligne n'est que de quelques dizaines de nanosecondes, la machine est suffisamment rapide pour que `track_io_timing` renvoie des résultats précis et sans ralentir la requête. C'est le cas sur presque toutes les machines et systèmes d'exploitation actuels, mais il y a parfois des surprises, par exemple dans certaines machines virtuelles ou selon la source de l'horloge système. Sinon, éviter d'activer `track_io_timing` sur un serveur de production. Sur une machine de test ou de formation, ce n'est pas un problème.

Exemple de résultat :

Un petit benchmark `pgbench` est lancé :

```
pgbench -c1 -R1 -T20 bench
```

Les plans d'exécution de l'ensemble des requêtes exécutées apparaissent alors dans les traces de l'instance.

```
2025-04-30 09:52:05.360 UTC [78782] LOG:  duration: 0.038 ms  plan:
Query Text: SELECT abalance FROM pgbench_accounts WHERE aid = 445977;
Index Scan using pgbench_accounts_3_pkey on pgbench_accounts_3 pgbench_accounts (cost=0.42..8.44 rows=1 width=4) (actual time=0.032..0.033 rows=1 loop=1)
Index Cond: (aid = 445977)
2025-04-30 09:52:05.360 UTC [78782] LOG:  duration: 0.237 ms  statement: SELECT abalance FROM pgbench_accounts WHERE aid = 445977;
2025-04-30 09:52:05.361 UTC [78782] LOG:  duration: 0.028 ms  plan:
Query Text: UPDATE pgbench_tellers SET tbalance = tbalance + 4529 WHERE tid = 33;
```

⁵<https://docs.postgresql.fr/current/pgtesttiming.html>

```

Update on pgbench_tellers (cost=0.00..1.63 rows=0 width=0) (actual time=0.027..0.027 rows=0 loops=1)
-> Seq Scan on pgbench_tellers (cost=0.00..1.63 rows=1 width=10) (actual time=0.015..0.017 rows=1 loops=1)
    Filter: (tid = 33)
    Rows Removed by Filter: 49
2025-04-30 09:52:05.361 UTC [78782] LOG: duration: 0.154 ms statement: UPDATE pgbench_tellers SET tbalance = tbalance + 4529 WHERE tid = 33;
2025-04-30 09:52:05.361 UTC [78782] LOG: duration: 0.012 ms plan:
Query Text: UPDATE pgbench_branches SET bbalance = bbalance + 4529 WHERE bid = 3;
Update on pgbench_branches (cost=0.00..1.06 rows=0 width=0) (actual time=0.011..0.011 rows=0 loops=1)
-> Seq Scan on pgbench_branches (cost=0.00..1.06 rows=1 width=10) (actual time=0.005..0.006 rows=1 loops=1)
    Filter: (bid = 3)
    Rows Removed by Filter: 4
2025-04-30 09:52:05.361 UTC [78782] LOG: duration: 0.101 ms statement: UPDATE pgbench_branches SET bbalance = bbalance + 4529 WHERE bid = 3;
2025-04-30 09:52:05.361 UTC [78782] LOG: duration: 0.011 ms plan:
Query Text: INSERT INTO pgbench_history (tid, bid, aid, delta, mtime) VALUES (33, 3, 445977, 4529, CURRENT_TIMESTAMP);
Insert on pgbench_history (cost=0.00..0.01 rows=0 width=0) (actual time=0.011..0.011 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=116) (actual time=0.003..0.004 rows=1 loops=1)
2025-04-30 09:52:05.361 UTC [78782] LOG: duration: 0.096 ms statement: INSERT INTO pgbench_history (tid, bid, aid, delta, mtime) VALUES (33, 3, 445977,
...

```

Utilisation pour des fonctions :

`auto_explain` est aussi un moyen de suivre les plans au sein de fonctions. Par défaut, un plan n'indique que les compteurs de blocs *hit*, *read*, *temp*... de l'appel global à la fonction.

Par exemple, on veut le plan à l'exécution de cette fonction en PL/pgSQL, qui récupère le solde le plus élevé dans la table `pgbench_accounts` :

```

CREATE OR REPLACE function f_max_balance() RETURNS int AS $$
DECLARE
    acct_balance int;
BEGIN
    SELECT max(abalance)
    INTO acct_balance
    FROM pgbench_accounts;
    RETURN acct_balance;
END;
$$ LANGUAGE plpgsql ;

```

Un `EXPLAIN ANALYZE` ne permet pas d'obtenir le plan de la requête de la fonction, car il est masqué au sein d'un nœud `Result` :

```
EXPLAIN (ANALYZE,VERBOSE) SELECT f_max_balance();
```

QUERY PLAN

```

--
Result (cost=0.00..0.26 rows=1 width=4) (actual time=37.047..37.048 rows=1 loops=1)
  Output: f_max_balance()
  Query Identifier: -5308051721018796792
  Planning Time: 0.059 ms
  Execution Time: 37.701 ms

```

(Exception : certaines fonctions très simples en SQL, et non PL/pgSQL, peuvent être intégrées dans la requête appelante et n'ont pas de plan propre.)

Par défaut, `auto_explain` ne va pas capturer plus d'information que la commande `EXPLAIN ANALYZE`. Le fichier log de l'instance capture le même plan lorsque la fonction est exécutée.

```

2025-04-30 09:49:17.829 UTC [78757] LOG: duration: 30.579 ms plan:
Query Text: EXPLAIN (ANALYZE,VERBOSE) SELECT f_max_balance();
Result (cost=0.00..0.26 rows=1 width=4) (actual time=30.576..30.577 rows=1 loops=1)

```

Output: f_max_balance()
 Buffers: shared hit=8199
 Query Identifier: -5308051721018796792

`auto_explain` permet de tracer les plans *dans* la fonction en activant le paramètre `auto_explain.log_nested_statements` de préférence uniquement dans la ou les sessions concernées :

```
\c bench
SET auto_explain.log_nested_statements = true;
SELECT f_max_balance();
```

Les deux plans (celui de la requête de la fonction, puis de la fonction) sont alors visibles dans les traces de l'instance :

```
2025-04-30 09:54:27.301 UTC [78790] LOG:  duration: 85.188 ms plan:
Query Text: SELECT max(abalance)
                                FROM pgbench_accounts
Finalize Aggregate (cost=13702.88..13702.89 rows=1 width=4) (actual time=85.182..85.184 rows=1 loops=1)
-> Gather (cost=13702.67..13702.88 rows=2 width=4) (actual time=85.175..85.178 rows=1 loops=1)
    Workers Planned: 2
    Workers Launched: 0
-> Partial Aggregate (cost=12702.67..12702.68 rows=1 width=4) (actual time=85.173..85.175 rows=1 loops=1)
    -> Parallel Append (cost=0.00..12181.84 rows=208332 width=4) (actual time=0.029..63.662 rows=500000 loops=1)
    -> Parallel Seq Scan on pgbench_accounts_1 (cost=0.00..3713.39 rows=98039 width=4) (actual time=0.028..14.006 rows=166667 loops=1)
    -> Parallel Seq Scan on pgbench_accounts_2 (cost=0.00..3713.39 rows=98039 width=4) (actual time=0.026..9.567 rows=166667 loops=1)
    -> Parallel Seq Scan on pgbench_accounts_3 (cost=0.00..3713.39 rows=98039 width=4) (actual time=0.007..9.687 rows=166666 loops=1)
2025-04-30 09:54:27.301 UTC [78790] CONTEXT:  SQL statement "SELECT max(abalance)
                                FROM pgbench_accounts"
PL/pgSQL function f_max_balance() line 5 at SQL statement
2025-04-30 09:54:27.302 UTC [78790] LOG:  duration: 85.520 ms plan:
Query Text: SELECT f_max_balance();
Result (cost=0.00..0.26 rows=1 width=4) (actual time=85.511..85.511 rows=1 loops=1)
2025-04-30 09:54:27.302 UTC [78790] LOG:  duration: 85.795 ms statement: SELECT f_max_balance();
```

Utilisation avec pgBadger :

pgBadger est capable de lire les plans tracés par `auto_explain`, de les intégrer à son rapport et d'inclure un lien vers explain.depesz.com⁶ pour une version plus lisible.

⁶<https://explain.depesz.com/>

1.5 PG_BUFFERCACHE



Qu'y a-t'il dans le cache de PostgreSQL ?

Fournit une vue :

- Pour chaque bloc de l'instance
 - fichier (donc objet) associé, base
 - fork (0 : table, 1 : FSM, 2 : VM)
 - numéro de bloc
 - `isdirty`
 - `usagecount` (0-5)
- Pour : utilisation du cache, *hot blocks*
- `pg_buffercache_evict()` (v17+)

`pg_buffercache` est une extension faisant partie de PostgreSQL, à installer dans chaque base au besoin.

Pour chaque entrée (bloc, par défaut de 8 ko) du cache disque de PostgreSQL, une vue nous fournit les informations suivantes :

- le fichier d'où provient ce bloc (donc indirectement le nom de la table ou de l'index...);
- le numéro de bloc dans ce fichier;
- si ce bloc est synchronisé avec le disque (`isdirty` à `false`);
- s'il est « sale » (*dirty*, modifié en mémoire mais non synchronisé sur disque);
- et si ce bloc a été utilisé récemment, selon la valeur de `usagecount`, de 0 (« n'est plus utilisé dernièrement ») à 5 (« récemment et fréquemment utilisé ») (voir les notions de *buffer pin* et *clocksweep*⁷).

Contenu du cache de PostgreSQL :

`pg_buffercache` permet donc d'abord de voir ce qui est dans le cache de PostgreSQL. À titre d'exemple, cette requête affiche les plus gros objets de la base en cours en mémoire cache (dont, ici, un index) :

```
CREATE EXTENSION pg_buffercache ;
```

```
SELECT c.relname,
       c.relkind,
       count(*) AS buffers,
       pg_size_pretty(count(*)*8192) as taille_mem
FROM   pg_buffercache b
LEFT OUTER JOIN pg_class c
  ON b.relfilenode = pg_relation_filenode(c.oid)
 AND b.reldatabase IN (0,(SELECT oid FROM pg_database
                          WHERE datname = current_database()))
GROUP BY c.relname, c.relkind
ORDER BY 3 DESC NULLS LAST
LIMIT 5 ;
```

⁷https://dali.bo/m3_html#:~:text=Buffer%20pin

relname	relkind	buffers	taille_mem
pgbench_accounts	r	724815	5663 MB
∅	∅	625618	4888 MB
pgbench_accounts_pkey	i	218864	1710 MB
pgbench_history	r	2809	22 MB
pgbench_tellers	r	149	1192 kB



Il faut être dans la bonne base de données pour que les noms des tables soient accessibles (d'où le `LEFT OUTER JOIN` et les noms à `NULL`, qui sont des blocs soit libres, soit utilisés par d'autres bases).

Il y a parfois des surprises. Par exemple, certains index sur des UUID v4 peuvent avoir tendance à monopoliser le cache⁸.

Rappelons que certains processus (`VACUUM`, `ANALYZE`, `COPY`, un parcours de table...) ont leur propre *ring buffer*⁹, et donc n'apparaîtront pas immédiatement dans `pg_buffercache`.

Hot blocks :

`pg_buffercache` permet aussi de déterminer les *hot blocks* de la base, ou d'avoir une idée un peu plus précise du bon dimensionnement du cache. Si aucun bloc n'atteint un `usagecount` de 5, le cache est manifestement trop petit : il n'est pas capable de détecter les pages devant impérativement rester en cache. Inversement, si vous avez énormément d'entrées à 0 et quelques pages avec des `usagecount` très élevés, toutes ces pages à 0 sont égales devant le mécanisme d'éviction du cache. Elles sont donc supprimées à peu près de la même façon que du cache du système d'exploitation. Le cache de PostgreSQL dans ce cas fait « double emploi » avec lui, et pourrait être réduit.

Attention toutefois avec les expérimentations sur les caches : il existe des effets de seuils. Un cache trop petit peut de la même façon qu'un cache trop grand avoir une grande fraction d'enregistrements avec un `usagecount` à 0. Par ailleurs, le cache bouge extrêmement rapidement par rapport à notre capacité d'analyse. Nous ne voyons qu'un instantané, qui peut ne pas refléter toute la réalité. Le cache peut aussi avoir un comportement complètement différent selon le moment de la journée.

`isdirty` indique si un buffer est synchronisé avec le disque ou pas. Il est intéressant de vérifier qu'une instance dispose en permanence d'un certain nombre de buffers pour lesquels `isdirty` vaut `false` et pour lesquels `usagecount` vaut 0. Ces blocs n'ont pas à être écrits sur disque, et ils n'ont pas été utilisés depuis un certain temps : ils peuvent être remplacés immédiatement par d'autres blocs. S'il n'y a pas assez de tels blocs, c'est le signe :

- que `shared_buffers` est probablement trop petit (il n'arrive pas à contenir les modifications);
- et/ou que le `background_writer` n'est pas assez agressif.

On peut suivre la quantité de blocs *dirty* et l'*usagecount* avec une requête de ce genre, ici juste après une petite mise à jour de la table `test` :

⁸https://dali.bo/s22_html#uuid-inconvénients

⁹https://dali.bo/m3_html#:~:text=Ring%20buffer

```

SELECT
    relname,
    isdirty,
    usagecount,
    pinning_backends,
    count(bufferid)
FROM pg_buffercache b
INNER JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE relname NOT LIKE 'pg\_%'
GROUP BY
    relname,
    isdirty,
    usagecount,
    pinning_backends
ORDER BY 1, 2, 3, 4 ;

```

relname	isdirty	usagecount	pinning_backends	count
brin_btree_idx	f	0	0	1
brin_btree_idx	f	1	0	7151
brin_btree_idx	f	2	0	3103
brin_btree_idx	f	3	0	10695
brin_btree_idx	f	4	0	141078
brin_btree_idx	f	5	0	2
brin_btree_idx	t	1	0	9
brin_btree_idx	t	2	0	1
brin_btree_idx	t	5	0	60
test	f	0	0	12371
test	f	1	0	6009
test	f	2	0	8466
test	f	3	0	1682
test	f	4	0	7393
test	f	5	0	112
test	t	1	0	1
test	t	5	0	267
test_pkey	f	1	0	173
test_pkey	f	2	0	27448
test_pkey	f	3	0	6644
test_pkey	f	4	0	10324
test_pkey	f	5	0	3420
test_pkey	t	1	0	57
test_pkey	t	3	0	81
test_pkey	t	4	0	116
test_pkey	t	5	0	15067

Évincer des blocs du cache :

Depuis PostgreSQL 17, la fonction `pg_buffercache_evict()` permet de libérer un bloc du cache. Dans les versions précédentes, nettoyer le cache impose de redémarrer PostgreSQL, ou de charger une très grosse table avec `pg_prewarm`. Cette fonctionnalité est utile par exemple pour tester des requêtes ou du paramétrage de manière reproductible, sans être perturbé par l'effet de cache.

Il faut appeler `pg_buffercache_evict()` pour chaque bloc, avec l'identifiant du bloc en paramètre, . Le code retour de la fonction est un booléen, à `true` en général, sauf si le buffer pointé n'existe pas, était utilisé (*pinned*) ou s'il a été sali (*dirtied*).

Pour supprimer du cache de PostgreSQL tous les blocs de la relation `pgbench_accounts` dans la base en cours :

```
SELECT c.relnamespace::regnamespace, c.relname,
       b.bufferid, pg_buffercache_evict(b.bufferid)
FROM pg_buffercache b
JOIN pg_class c
ON b.relfilenode = pg_relation_filenode(c.oid) AND
   b.reldatabase IN (0, (SELECT oid FROM pg_database
                        WHERE datname = current_database()))
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relname = 'pgbench_accounts';
```

```
...
public      | pgbench_accounts | 1572860 | t
public      | pgbench_accounts | 1572861 | t
public      | pgbench_accounts | 1572862 | t
public      | pgbench_accounts | 1572863 | t
public      | pgbench_accounts | 1572864 | t
(1564533 lignes)
```

ou de manière plus synthétique :

```
SELECT c.relnamespace::regnamespace, c.relname,
       pg_buffercache_evict(b.bufferid), count(b.bufferid)
FROM pg_buffercache b
JOIN pg_class c
ON b.relfilenode = pg_relation_filenode(c.oid) AND
   b.reldatabase IN (0, (SELECT oid FROM pg_database
                        WHERE datname = current_database()))
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relname = 'pgbench_accounts'
GROUP BY 1,2,3 ;
```

relnamespace	relname	pg_buffercache_evict	count
public	pgbench_accounts	f	1
public	pgbench_accounts	t	1292638

À l'appel suivant, la table devra être relue hors du cache de PostgreSQL (mention *read*) :

```
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT * FROM pgbench_accounts ;
```

QUERY PLAN

```
-----
Seq Scan on pgbench_accounts (actual time=0.199..27994.323 rows=100000000 loops=1)
  Buffers: shared read=1646640
  I/O Timings: shared read=18049.770
Planning Time: 0.035 ms
Execution Time: 32228.828 ms
```

Cache de Linux :

Le cache de Linux est complémentaire de celui de PostgreSQL. Pour des tests, il ne faut pas oublier de le purger aussi :

```
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Pour étudier le comportement du cache de Linux, il existe l'extension `pg_stat_kcache`¹⁰.

¹⁰https://github.com/powa-team/pg_stat_kcache

1.6 PG_PREWARM



— Charge des blocs en cache :

```
-- cache de PG
SELECT pg_prewarm ('pgbench_accounts', 'buffer') ;

-- cache de Linux (asynchrone)
SELECT pg_prewarm ('pgbench_accounts', 'prefetch') ;
-- cache (tous OS)
SELECT pg_prewarm ('pgbench_accounts', 'read') ;
```

- Ne pas oublier les index!
- N'interdit pas l'éviction
- Récupération du cache au redémarrage (v11)
- avec un petit paramétrage

Grâce à l'extension `pg_prewarm`, intégrée à PostgreSQL, il est possible de pré-charger une table ou d'autres objets dans la mémoire de PostgreSQL, ou celle du système d'exploitation, pour améliorer les performances par la suite.

Par exemple, on charge la table `pgbench_accounts` dans le cache de PostgreSQL ainsi, et on le vérifie avec `pg_buffercache` :

```
CREATE EXTENSION IF NOT EXISTS pg_prewarm ;

SELECT pg_prewarm ('pgbench_accounts', 'buffer') ;
```

```
pg_prewarm
-----
163935
```

La valeur retournée correspond aux blocs chargés.

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache ;

SELECT c.relname, count(*) AS buffers, pg_size_pretty(count(*)*8192) as taille_mem
FROM pg_buffercache b INNER JOIN pg_class c
ON b.relfilenode = pg_relation_filenode(c.oid)
GROUP BY c.relname ;
```

relname	buffers	taille_mem
pgbench_accounts	163935	1281 MB

Il faut rappeler qu'une table ne se résume pas à ses données! Il est au moins aussi intéressant de récupérer les index de la table en question :

```
SELECT pg_prewarm ('pgbench_accounts_pkey', 'buffer');
```

Si le cache de PostgreSQL ne suffit pas, celui du système peut être aussi préchargé :

```
SELECT pg_prewarm ('pgbench_accounts_pkey', 'read');  
SELECT pg_prewarm ('pgbench_accounts_pkey', 'prefetch'); -- à préférer sur Linux
```

Charger une table en cache ne veut pas dire qu'elle va y rester ! Si les blocs chargés ne sont pas utilisés, ils seront évincés quand PostgreSQL aura besoin de faire de la place dans le cache, comme n'importe quels autres blocs.

Automatisation :

Cette extension peut sauvegarder le contenu du cache à intervalles réguliers ou lors de l'arrêt (propre) de PostgreSQL et le restaurer au redémarrage. Pour cela, paramétrer ceci :

```
shared_preload_libraries = 'pg_prewarm'  
pg_prewarm.autoprewarm = on  
pg_prewarm.autoprewarm_interval = '5min'
```

Les blocs concernés sont sauvés dans un fichier `autoprewarm.blocks` dans le répertoire PGDATA. Un *worker* nommé `autoprewarm leader` apparaîtra.

L'intérêt est de réduire énormément la phase de rechargement en cache des données actives après un redémarrage, accidentel ou non. En effet, une grosse base très active et aux disques un peu lents peut mettre longtemps à re-remplir son cache et à retrouver des performances acceptables. De plus, ne seront rechargées que les données en cache précédemment, donc à priori les parties de tables réellement actives.

Autres possibilités :

La documentation ¹¹ décrit également comment charger :

- d'autres parties de la table comme la *visibility map* ;
- certains blocs précis.

¹¹<https://docs.postgresql.fr/current/pgprewarm.html>

1.7 LANGAGES PROCÉDURAUX



- Procédures & fonctions en différents langages
- Par défaut : SQL, C et PL/pgSQL
- Extensions officielles : Perl, Python
- Mais aussi Java, Ruby, Javascript...
- Intérêts : fonctionnalités, performances

Les langages officiellement supportés par le projet sont :

- PL/pgSQL;
- PL/Perl¹²;
- PL/Python¹³;
- PL/Tcl.

Voici une liste non exhaustive des langages procéduraux disponibles, à différents degrés de maturité :

- PL/sh¹⁴;
- PL/R¹⁵;
- PL/Java¹⁶;
- PL/lolcode;
- PL/Scheme;
- PL/PHP;
- PL/Ruby;
- PL/Lua¹⁷;
- PL/pgPSM;
- PL/v8¹⁸ (Javascript).

Le wiki PostgreSQL contient un tableau des langages supportés¹⁹.

Pour qu'un langage soit utilisable, il doit être activé au niveau de la base où il sera utilisé. Les trois langages activés par défaut sont le C, le SQL et le PL/pgSQL. Les autres doivent être ajoutés à partir des paquets de la distribution ou du PGDG, ou compilés à la main, puis l'extension installée dans la base :

```
CREATE EXTENSION plperl ;
CREATE EXTENSION plpython3u ;
-- etc.
```

¹²<https://docs.postgresql.fr/current/plperl.html>

¹³<https://docs.postgresql.fr/current/plpython.html>

¹⁴<https://github.com/petere/plsh>

¹⁵<https://github.com/postgres-plr/plr>

¹⁶<https://tada.github.io/pljava/>

¹⁷<https://github.com/pllua/pllua>

¹⁸<https://github.com/plv8/plv8>

¹⁹https://wiki.postgresql.org/wiki/PL_Matrix

Ces fonctions peuvent être utilisées dans des index fonctionnels et des triggers comme toute fonction SQL ou PL/pgSQL.

Chaque langage a ses avantages et inconvénients. Par exemple, PL/pgSQL est très simple à apprendre mais n'est pas performant quand il s'agit de traiter des chaînes de caractères. Pour ce traitement, il est souvent préférable d'utiliser PL/Perl, voire PL/Python. Évidemment, une routine en C aura les meilleures performances mais sera beaucoup moins facile à coder et à maintenir, et ses bugs seront susceptibles de provoquer un plantage du serveur.

Par ailleurs, les procédures peuvent s'appeler les unes les autres quel que soit le langage. S'ajoute l'intérêt de ne pas avoir à réécrire en PL/pgSQL des fonctions existantes dans d'autres langages ou d'accéder à des modules bien établis de ces langages.

1.7.1 Avantages & inconvénients



- PL/pgSQL plus performant pour l'accès aux données
- Chaque langage a son point fort
- Performances :
 - latence (pas d'allers-retours)
 - accès aux données depuis les fonctions
 - bibliothèques de chaque langage
 - index
- Langages *trusted* / *untrusted*

Il est courant de considérer que la logique métier (les fonctions) doit être intégralement dans l'appli-catif, et pas dans la base de données. Même si l'on adopte ce point de vue, il faut savoir faire des ex-ceptions pour prendre en compte les performances : une fonction, en PL/pgSQL ou un autre langage, exécutée dans la base de données économisera des aller-retours entre la base et le serveur applicatif, ce qui peut avoir un impact énorme (latence due à de nombreux ordres, ou durée de transfert des résultats intermédiaires).

Une fonction en Perl ou Python complexe peut servir aussi de critère d'indexation, pour des gains parfois énormes.

Le PL/pgSQL²⁰ est le mieux intégré des langages (avec le C), D'autres langages peuvent subir une pé-nalité due à la communication avec l'interpréteur (car c'est bien celui présent sur le serveur qui est utilisé). Cependant, ils peuvent apporter des fonctionnalités qui manquent à PostgreSQL : PL/R, bi-bliothèques numériques NumPy et Scipy de Python...

Pour des raisons de sécurité, on distingue des langages *trusted* et *untrusted*. Un langage *trusted* est disponible pour tous les utilisateurs de la base, n'autorise pas l'accès à des données normalement inaccessibles à l'utilisateur, mais quelques fonctionnalités ont pu être supprimées (interaction avec l'environnement notamment). Un langage *untrusted* n'a pas ces limites et les fonctions ne peuvent

²⁰<https://www.postgresql.org/docs/current/plpgsql-overview.html>

être créées que par un super-utilisateur. PL/pgSQL est *trusted*. PL/Python n'existe qu'en *untrusted* (l'extension pour la version 3 se nomme `plpython3u`). PL/Perl existe dans les deux versions (extensions `plperl` et `plperlu`).

1.8 HLL



- `COUNT(DISTINCT)` est notoirement lent
- Principe d'HyperLogLog :
 - travail sur des hachages, avec perte
 - estimation statistique
 - non exact, mais beaucoup plus rapide
- Exemple :

```
SELECT mois, hll_cardinality(hll_add_agg(hll_hash_text( id )))
FROM voyages ;
```

- Type `hll` pour pré-agréger des données

Les décomptes de valeurs distinctes sont une opération assez courante dans certains domaines : décompte de visiteurs distincts d'un site web ou d'un lieu, de patients d'un hôpital, de voyageurs, etc. Or `COUNT(DISTINCT)` est notoirement lent quand on fait face à un grand nombre de valeurs distinctes, à cause de la déduplication des valeurs, du maintien d'un espace pour le décompte, du besoin fréquent de fichiers temporaires...

Le principe de HyperLogLog est de ne pas opérer de calculs exacts mais de compiler un hachage des données rencontrées, avec perte, et donc beaucoup de manière plus compacte; puis d'étudier la répartition statistique des valeurs rencontrées, et d'en déduire la volumétrie approximative. En effet, dans beaucoup de contexte, il n'est pas forcément utile de connaître le nombre *exact* de clients, de passagers... Une approximation peut répondre à beaucoup de besoins. En fonction de l'imprécision acceptée, on peut économiser beaucoup de mémoire et de temps (un gain d'un facteur supérieur à 10 est fréquent).

Une extension dédiée existe, à présent maintenue par Citusdata. Le source est sur Github²¹, et on trouvera les paquets dans les dépôts communautaires habituels.

La bibliothèque doit être préchargée dans chaque session pour être exécuté par l'optimiseur pour influencer les plans générés :

```
shared_preload_libraries = 'hll'
```

Puis charger l'extension dans la base concernée :

```
CREATE EXTENSION hll ;
```

On peut alors immédiatement remplacer un `COUNT(DISTINCT id)` par cet équivalent :

```
SELECT mois, hll_cardinality(hll_add_agg(hll_hash_text( id )))
FROM matable ;
```

²¹<https://github.com/citusdata/postgresql-hll>

Concrètement, l'identifiant à trier est haché (il y a une fonction dédiée par type). Puis ces hachages sont agrégés en un ensemble par la fonction `hll_add_agg()`. Ensuite, la fonction `hll_cardinality()` estime le nombre de valeurs distinctes originales à partir de cet ensemble.

Le paramétrage par défaut est déjà pertinent pour des cardinalités jusqu'au billion (10^{12}) d'après la documentation²², avec une erreur de l'ordre du pour cent. La précision de l'estimation peut être ajustée de manière générale, ou bien comme paramètre à la fonction de création de l'ensemble, comme dans ces exemples (ici avec les valeurs par défaut) :

```
SELECT hll_set_defaults(11, 5, -1, 1) ;
```

```
SELECT hll_cardinality(hll_add_agg(hll_hash_text( id ), 11, 5, -1, 1 ))  
FROM matable ;
```

Les deux premiers paramètres sont les plus importants : le nombre de registres utilisés (de 4 à 31, chaque incrément de 1 doublant la taille mémoire requise), et la taille des registres en bits (de 1 à 8). Des valeurs trop grandes risquent de rendre l'estimation inutilisable (résultat `NaN`).

Dans le monde décisionnel, il est fréquent de créer des tables d'agrégat avec des résultats pré-calculés sur un jour ou un mois. Cela ne fonctionne que partiellement pour des `COUNT(DISTINCT)` : par exemple, on ne peut sommer le nombre de voyageurs distincts de chaque mois pour calculer celui sur l'année, ce sont peut-être les mêmes clients toute l'année. L'extension apporte donc aussi un type `hll` destiné à stocker des résultats agrégés issus d'un appel à `hll_add_agg()`. On agrège le contenu de ces champs `hll` avec la fonction `hll_union_agg()`, et on peut procéder à l'estimation sur l'ensemble avec `hll_cardinality`.

²²<https://github.com/citusdata/postgresql-hll>

1.9 QUIZ



https://dali.bo/x2_quiz

1.10 TRAVAUX PRATIQUES

La version en ligne des solutions de ces TP est disponible sur https://dali.bo/x2_solutions.

1.10.1 Indexation de pattern avec les varchar_patterns et pg_trgm



But : Indexer des patterns avec les varchar_patterns et pg_trgm

Ces exercices nécessitent une base contenant une quantité de données importante.

On utilisera donc le contenu de livres issus du projet Gutenberg. La base **gutenberg** est disponible en deux versions : complète sur https://dali.bo/tp_gutenberg (dump de 0,5 Go, table de 21 millions de lignes dans 3 Go) ou https://dali.bo/tp_gutenberg10 pour un extrait d'un dixième. Le dump peut se restaurer par exemple dans une nouvelle base, et contient juste une table nommée `textes`.

```
curl -kL https://dali.bo/tp_gutenberg -o /tmp/gutenberg.dmp
createdb gutenberg
pg_restore -d gutenberg /tmp/gutenberg.dmp
# le message sur le schéma public existant est normale
rm -- /tmp/gutenberg.dmp
```

Pour obtenir des plans plus lisibles, on désactive JIT et parallélisme :

```
SET jit TO off;
SET max_parallel_workers_per_gather TO 0;
```

Créer un index simple sur la colonne `contenu` de la table.

Rechercher un enregistrement commençant par « comme disent » : l'index est-il utilisé ?

Créer un index utilisant la classe `text_pattern_ops`. Refaire le test.

On veut chercher les lignes finissant par « Et vivre ». Indexer `reverse(contenu)` et trouver les lignes.

Installer l'extension `pg_trgm`, puis créer un index GIN spécialisé de recherche dans les chaînes. Rechercher toutes les lignes de texte contenant « Valjean » de façon sensible à la casse, puis insensible.

Si vous avez des connaissances sur les expression rationnelles, utilisez aussi ces trigrammes pour des recherches plus avancées. Les opérateurs sont :

opérateur	fonction
~	correspondance sensible à la casse
~*	correspondance insensible à la casse
!~	non-correspondance sensible à la casse
!~*	non-correspondance insensible à la casse

Rechercher toutes les lignes contenant « Fantine » OU « Valjean » : on peut utiliser une expression rationnelle.

Rechercher toutes les lignes mentionnant à la fois « Fantine » ET « Valjean ». Une formulation d'expression rationnelle simple est « Fantine puis Valjean » ou « Valjean puis Fantine ».

1.10.2 auto_explain



But : Capturer les plans d'exécutions automatiquement avec auto_explain

Installer le module `auto_explain` (documentation : <https://docs.postgresql.fr/current/auto-explain.html>).

Exécuter des requêtes sur n'importe quelle base de données, et inspecter les traces générées.

Passer le niveau de messages de sa session (`client_min_messages`) à `log`.

1.10.3 pg_stat_statements



But : Analyser les performances des requêtes avec pg_stat_statements

— `pg_stat_statements` nécessite une bibliothèque préchargée. La positionner dans le fichier `postgresql.conf`, redémarrer PostgreSQL et créer l'extension.

- Inspecter le contenu de l'extension `pg_stat_statements` (`\dx` et `\dx+`).
- Vérifier que le serveur est capable d'activer la mesure de la durée des entrées-sorties avec `pg_test_timing`. Puis l'activer (`track_io_timing`), sans oublier de redémarrer PostgreSQL.
- Depuis un autre terminal, créer une base **pgbench** (si pas déjà disponible), l'initialiser (même si elle existait), et lancer une activité dessus en arrière-plan :

```
# en tant qu'utilisateur postgres
createdb -e pgbench
/usr/pgsql-16/bin/pgbench -i -s135 pgbench
/usr/pgsql-16/bin/pgbench -c5 -j1 pgbench -T 600 -P1
```

- Dans la vue `pg_stat_statements`, récupérer les 5 requêtes les plus gourmandes en temps cumulé sur l'instance et leur nombre de lignes.

Quelle est la requête générant le plus d'écritures directes sur disques (*written*) ? Et en temps d'écriture ?

Quel est le *hit ratio* des requêtes les plus fréquentes ?

1.10.4 PL/Python, import de page web et compression



But : Importer et stocker une page web au format compressé avec PL/Python

Sur la base du code suivant en python 3 utilisant un des modules standard (documentation : <https://docs.python.org/3/library/urllib.request.html>), créer une fonction PL/Python récupérant le code HTML d'une page web avec un simple `SELECT pageweb('https://www.postgresql.org/') :`

```
import urllib.request
f = urllib.request.urlopen('https://www.postgresql.org/')
print (f.read().decode('utf-8'))
```

Stocker le résultat dans une table.

Puis stocker cette page en compression maximale dans un champ `bytea`, en passant par une fonction python inspirée du code suivant (documentation : <https://docs.python.org/3/library/bz2.html>) :

```
import bz2
compressed_data = bz2.compress(data, compresslevel=9)
```

Écrire la fonction de décompression avec la fonction python `bz2.decompress`.

Utiliser ensuite `convert_from( bytea, 'UTF8')` pour récupérer un `text`.

1.10.5 PL/Perl et comparaison de performances



But : Exécuter un traitement performant avec PL/Perl

Ce TP s'inspire d'un billet de blog de Daniel Vérité²³, qui a publié le code des fonctions sur le wiki PostgreSQL sous licence PostgreSQL. Le principe est d'implémenter un remplacement en masse de nombreuses chaînes de caractères par d'autres. Une fonction codée en PL/perl peut se révéler plus rapide qu'une autre en PL/pgSQL.

Il utilise la base de données contenant des livres issus du projet Gutenberg, dans sa version complète qui contient *Les Misérables* de Victor Hugo. La base **gutenberg** est disponible en deux versions : complète sur https://dali.bo/tp_gutenberg (dump de 0,5 Go, table de 21 millions de lignes dans 3 Go) ou https://dali.bo/tp_gutenberg10 pour un extrait d'un dixième. Le dump peut se restaurer par exemple dans une nouvelle base, et contient juste une table nommée `textes`.

```
curl -kL https://dali.bo/tp_gutenberg -o /tmp/gutenberg.dmp
createdb gutenberg
pg_restore -d gutenberg /tmp/gutenberg.dmp
# le message sur le schéma public existant est normale
rm -- /tmp/gutenberg.dmp
```

Créer la fonction `multi_replace` en PL/pgSQL à partir du wiki PostgreSQL : https://wiki.postgresql.org/wiki/Multi_Replace_plpgsql

Récupérer la fonction en PL/perl sur le même wiki : https://wiki.postgresql.org/wiki/Multi_Replace_Perl.

Vérifier que les deux fonctions ont le même nom mais des types de paramètres différents.

²³<https://blog-postgresql.verite.pro/2020/01/22/multi-replace.html>

Le test va consister à transposer tous les noms et lieux des *Misérables* de Victor Hugo dans une version américaine :

- charger la base du projet Gutenberg si elle n'est pas déjà en place.
- créer une table `miserables` reprenant tous les livres dont le titre commence par « Les misérables ».

Tester le bon fonctionnement avec ces requêtes :

```
SELECT multi_replace (contenu, '{"Valjean":"Valjohn", "Cosette":"Lucy"}'::jsonb)
FROM miserables
WHERE contenu ~ '(Valjean|Cosette)' LIMIT 5 ;
```

```
SELECT multi_replace(contenu, '{Valjean,Cosette}', '{Valjohn, Lucy}' )
FROM miserables
WHERE contenu ~ '(Valjean|Cosette)' LIMIT 5 ;
```

Pour faciliter la modification, prévoir une table pour stocker les critères :

```
CREATE TABLE remplacement (j jsonb, old_t text[], new_t text[]) ;
```

Insérer par exemple les données suivantes :

```
INSERT INTO remplacement (j)
SELECT '{"Valjean":"Valjohn", "Jean Valjean":"John Valjohn",
"Cosette":"Lucy", "Fantine":"Fanny", "Javert":"Green",
"Thénardier":"Thenardy", "Éponine":"Sharon", "Azelma":"Azealia",
"Marius":"Marc", "Gavroche":"Garry", "Enjolras":"Joker",
"Notre-Dame":"Empire State Building", "Victor Hugo":"Victor Hugues",
"Hugo":"Hugues", "Fauchelevent":"Dropwind", "Bouchart":"Butcher",
"Célestine":"Celeste", "Mabeuf":"Myoax", "Leblanc":"White",
"Combeferre":"Combiron", "Magloire":"Glory",
"Gillenormand":"Jillnorthman", "France":"États-Unis",
"Paris":"New York", "Louis Philippe":"Andrew Jackson" }'::jsonb ;
```

Copier le contenu sous forme de tableau de caractères dans les autres champs :

```
UPDATE remplacement
SET old_t = noms_old , new_t = noms_new
FROM ( SELECT array_agg (key) AS noms_old, array_agg (value) AS noms_new
FROM (
    SELECT (jsonb_each_text (j)).* FROM remplacement
    ) j1
    ) j2 ;
```

Comparer la performance des deux fonctions suivantes :

```
\pset pager off
-- fonction en PL/perl
EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu, (SELECT j FROM remplacement))
FROM miserables ;

-- fonction en PL/pgSQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu,
                      (SELECT old_t FROM remplacement),
                      (SELECT new_t FROM remplacement) )
FROM miserables ;
```

1.10.6 hll



But : Estimer le nombre de valeurs distinctes plus rapidement avec hll

- Installer l'extension `hll` dans la base de données de test :
- le paquet est `hll_17` ou `postgresql-17-hll` (adapter le numéro à votre version de PostgreSQL) selon la distribution ;
- l'extension se nomme `hll` ;
- elle nécessite d'être préalablement déclarée dans `shared_preload_libraries`.

- Créer un jeu de données simulant des voyages en transport en commun, par passager selon la date :

```
CREATE TABLE voyages
(voyage_id      bigint GENERATED ALWAYS AS IDENTITY,
 passager_id    text,
 d              date
) ;

INSERT INTO voyages (passager_id, d)
SELECT sem+mod(i, sem+1) || '-' || mod(i,77777) AS passager_id, d
FROM generate_series (0,51) sem,
     LATERAL
     (SELECT i,
      '2019-01-01'::date + sem * interval '7 days' + i * interval '2s' AS d
     FROM generate_series (1,
      (case when sem in (31,32,33) then 0 else 22 end +abs(30-sem))*5000 ) i
     ) j
;
```

- Activer l’affichage du temps (`timing`).
 - Désactiver JIT et le parallélisme.
 - Passer la mémoire de tri à 1 Go.
 - Précharger la table dans le cache de PostgreSQL.
-
- Calculer, par mois, le nombre exact de voyages et de passagers **distincts**.
 - Dans le plan de la requête, chercher où est perdu le temps.
-
- Calculer, pour l’année, le nombre exact de voyages et de passagers **distincts**.
-
- Recompter les passagers dans les deux cas en remplaçant le `COUNT(DISTINCT)` par cette expression : `hll_cardinality(hll_add_agg(hll_hash_text(passager_id))::int)`
-
- Réexécuter les requêtes après modification du paramétrage de `hll` :

```
SELECT hll_set_defaults(17, 5, -1, 0);
```
-
- Créer une table d’agrégat par mois avec un champ d’agrégat `hll` et la remplir.
-
- À partir de cette table d’agrégat :
- calculer le nombre moyen mensuel de passagers distincts,
 - recalculer le nombre de passagers distincts sur l’année à partir de cette table d’agrégat.
-
- Avec une fonction de fenêtrage sur `hll_union_agg`, calculer une moyenne glissante sur 3 mois du nombre de passagers distincts.

1.11 TRAVAUX PRATIQUES (SOLUTIONS)

1.11.1 Indexation de pattern avec les varchar_patterns et pg_trgm

Créer un index simple sur la colonne `contenu` de la table.

```
CREATE INDEX ON textes(contenu);
```

Il y aura une erreur si la base **gutenberg** est dans sa version complète, un livre de Marcel Proust dépasse la taille indexable maximale :

```
ERROR: index row size 2968 exceeds maximum 2712 for index "textes_contenu_idx"
ASTUCE : Values larger than 1/3 of a buffer page cannot be indexed.
Consider a function index of an MD5 hash of the value, or use full text indexing.
```

Pour l'exercice, on supprime ce livre avant d'indexer la colonne :

```
DELETE FROM textes where livre = 'Les Demi-Vierges, Prévost, Marcel';
CREATE INDEX ON textes(contenu);
```

Rechercher un enregistrement commençant par « comme disent » : l'index est-il utilisé ?

Le plan exact peut dépendre de la version de PostgreSQL, du paramétrage exact, d'éventuelles modifications à la table. Dans beaucoup de cas, on obtiendra :

```
SET jit TO off;
SET max_parallel_workers_per_gather TO 0;
VACUUM ANALYZE textes;

EXPLAIN ANALYZE SELECT * FROM textes WHERE contenu LIKE 'comme disent%';
```

QUERY PLAN

```
-----
Seq Scan on textes (cost=0.00..669657.38 rows=1668 width=124)
    (actual time=305.848..6275.845 rows=47 loops=1)
    Filter: (contenu ~~ 'comme disent% '::text)
    Rows Removed by Filter: 20945503
    Planning Time: 1.033 ms
    Execution Time: 6275.957 ms
```

C'est un `Seq Scan` : l'index n'est pas utilisé !

Dans d'autres cas, on aura ceci (avec PostgreSQL 12 et la version complète de la base ici) :

```
EXPLAIN ANALYZE SELECT * FROM textes WHERE contenu LIKE 'comme disent%';
```

QUERY PLAN

```
-----
Index Scan using textes_contenu_idx on textes (...)
    Index Cond: (contenu ~~ 'comme disent% '::text)
    Rows Removed by Index Recheck: 110
    Buffers: shared hit=28 read=49279
    I/O Timings: read=311238.192
    Planning Time: 0.352 ms
    Execution Time: 313481.602 ms
```

C'est un `Index Scan` mais il ne faut pas crier victoire : l'index est parcouru entièrement (50 000 blocs!). Il ne sert qu'à lire toutes les valeurs de `contenu` en lisant moins de blocs que par un `Seq Scan` de la table. Le choix de PostgreSQL entre lire cet index et lire la table dépend notamment du paramétrage et des tailles respectives.

Le problème est que l'index sur `contenu` utilise la collation `C` et non la collation par défaut de la base, généralement `en_US.UTF-8` ou `fr_FR.UTF-8`. Pour contourner cette limitation, PostgreSQL fournit deux classes d'opérateurs : `varchar_pattern_ops` pour `varchar` et `text_pattern_ops` pour `text`.

Créer un index utilisant la classe `text_pattern_ops`. Refaire le test.

```
DROP INDEX textes_contenu_idx;
CREATE INDEX ON textes(contenu text_pattern_ops);

EXPLAIN (ANALYZE,BUFFERS)
SELECT * FROM textes WHERE contenu LIKE 'comme disent%';
```

QUERY PLAN

```
-----
Index Scan using textes_contenu_idx1 on textes
      (cost=0.56..8.58 rows=185 width=130)
      (actual time=0.530..0.542 rows=4 loops=1)
    Index Cond: ((contenu ~>= 'comme disent'::text)
      AND (contenu ~<= 'comme disenu'::text))
    Filter: (contenu ~ 'comme disent%'::text)
    Buffers: shared hit=4 read=4
Planning Time: 1.112 ms
Execution Time: 0.618 ms
```

On constate que comme l'ordre choisi est l'ordre ASCII, l'optimiseur sait qu'après « comme disent », c'est « comme disenu » qui apparaît dans l'index.

Noter que `Index Cond` contient le filtre utilisé pour l'index (réexprimé sous forme d'inégalités en collation `C`) et `Filter` un filtrage des résultats de l'index.

On veut chercher les lignes finissant par « Et vivre ». Indexer `reverse(contenu)` et trouver les lignes.

Cette recherche n'est possible avec un index B-Tree qu'en utilisant un index sur fonction :

```
CREATE INDEX ON textes(reverse(contenu) text_pattern_ops);
```

Il faut ensuite utiliser ce `reverse` systématiquement dans les requêtes :

```
EXPLAIN (ANALYZE)
SELECT * FROM textes WHERE reverse(contenu) LIKE reverse('%Et vivre') ;
```

QUERY PLAN

```
-----
Index Scan using textes_reverse_idx on textes
      (cost=0.56..377770.76 rows=104728 width=123)
```

```

              (actual time=0.083..0.098 rows=2 loops=1)
Index Cond: ((reverse(contenu) ~>= 'erviv tE'::text)
              AND (reverse(contenu) ~<= 'erviv tF'::text))
Filter: (reverse(contenu) ~~ 'erviv tE%'::text)
Planning Time: 1.903 ms
Execution Time: 0.421 ms

```

On constate que le résultat de `reverse(contenu)` a été directement utilisé par l'optimiseur. La requête est donc très rapide. On peut utiliser une méthode similaire pour la recherche insensible à la casse, en utilisant `lower()` ou `upper()`.

Toutefois, ces méthodes ne permettent de filtrer qu'au début ou à la fin de la chaîne, ne permettent qu'une recherche sensible ou insensible à la casse, mais pas les deux simultanément, et imposent aux développeurs de préciser `reverse`, `lower`, etc. partout.

Installer l'extension `pg_trgm`, puis créer un index GIN spécialisé de recherche dans les chaînes. Rechercher toutes les lignes de texte contenant « Valjean » de façon sensible à la casse, puis insensible.

Pour installer l'extension `pg_trgm` :

```
CREATE EXTENSION pg_trgm;
```

Pour créer un index GIN sur la colonne `contenu` :

```
CREATE INDEX idx_textes_trgm ON textes USING gin (contenu gin_trgm_ops);
```

Recherche des lignes contenant « Valjean » de façon sensible à la casse :

```
EXPLAIN (ANALYZE)
SELECT * FROM textes WHERE contenu LIKE '%Valjean%' ;
```

QUERY PLAN

```

-----
Bitmap Heap Scan on textes  (cost=77.01..6479.68 rows=1679 width=123)
    (actual time=11.004..14.769 rows=1213 loops=1)
    Recheck Cond: (contenu ~~ '%Valjean%'::text)
    Rows Removed by Index Recheck: 1
    Heap Blocks: exact=353
    -> Bitmap Index Scan on idx_textes_trgm
        (cost=0.00..76.59 rows=1679 width=0)
        (actual time=10.797..10.797 rows=1214 loops=1)
        Index Cond: (contenu ~~ '%Valjean%'::text)
Planning Time: 0.815 ms
Execution Time: 15.122 ms

```

Puis insensible à la casse :

```
EXPLAIN ANALYZE SELECT * FROM textes WHERE contenu ILIKE '%Valjean%';
```

QUERY PLAN

```

-----
Bitmap Heap Scan on textes  (cost=77.01..6479.68 rows=1679 width=123)
    (actual time=13.135..23.145 rows=1214 loops=1)

```

```

Recheck Cond: (contenu ~* '%Valjean%':text)
Heap Blocks: exact=353
-> Bitmap Index Scan on idx_textes_trgm
      (cost=0.00..76.59 rows=1679 width=0)
      (actual time=12.779..12.779 rows=1214 loops=1)
    Index Cond: (contenu ~* '%Valjean%':text)
Planning Time: 2.047 ms
Execution Time: 23.444 ms

```

On constate que l'index a été nettement plus long à créer, et que la recherche est plus lente. La contrepartie est évidemment que les trigrammes sont infiniment plus souples. On constate aussi que le `LIKE` a dû encore filtrer 1 enregistrement après le parcours de l'index : en effet l'index trigramme est insensible à la casse, il ramène donc trop d'enregistrements, et une ligne avec « VALJEAN » a dû être filtrée.

Rechercher toutes les lignes contenant « Fantine » OU « Valjean » : on peut utiliser une expression rationnelle.

```
EXPLAIN ANALYZE SELECT * FROM textes WHERE contenu ~ 'Valjean|Fantine';
```

QUERY PLAN

```

-----
Bitmap Heap Scan on textes  (cost=141.01..6543.68 rows=1679 width=123)
    (actual time=159.896..174.173 rows=1439 loops=1)
    Recheck Cond: (contenu ~ 'Valjean|Fantine':text)
    Rows Removed by Index Recheck: 1569
    Heap Blocks: exact=1955
    -> Bitmap Index Scan on idx_textes_trgm
          (cost=0.00..140.59 rows=1679 width=0)
          (actual time=159.135..159.135 rows=3008 loops=1)
        Index Cond: (contenu ~ 'Valjean|Fantine':text)
Planning Time: 2.467 ms
Execution Time: 174.284 ms

```

Rechercher toutes les lignes mentionnant à la fois « Fantine » ET « Valjean ». Une formulation d'expression rationnelle simple est « Fantine puis Valjean » ou « Valjean puis Fantine ».

```
EXPLAIN ANALYZE SELECT * FROM textes
WHERE contenu ~ '(Valjean.*Fantine)|(Fantine.*Valjean)';
```

QUERY PLAN

```

-----
Bitmap Heap Scan on textes  (cost=141.01..6543.68 rows=1679 width=123)
    (actual time=26.825..26.897 rows=8 loops=1)
    Recheck Cond: (contenu ~ '(Valjean.*Fantine)|(Fantine.*Valjean)':text)
    Heap Blocks: exact=6
    -> Bitmap Index Scan on idx_textes_trgm
          (cost=0.00..140.59 rows=1679 width=0)
          (actual time=26.791..26.791 rows=8 loops=1)
        Index Cond: (contenu ~ '(Valjean.*Fantine)|(Fantine.*Valjean)':text)
Planning Time: 5.697 ms
Execution Time: 26.992 ms

```

1.11.2 auto_explain

Installer le module `auto_explain` (documentation : <https://docs.postgresql.fr/current/auto-explain.html>).

Dans le fichier `postgresql.conf`, chargement du module et activation globale pour *toutes* les requêtes (ce qu'on évitera de faire en production) :

```
shared_preload_libraries = 'auto_explain'  
auto_explain.log_min_duration = 0
```

Redémarrer PostgreSQL.

Exécuter des requêtes sur n'importe quelle base de données, et inspecter les traces générées.

Le plan de la moindre requête (même un `\d+`) doit apparaître dans la trace.

Passer le niveau de messages de sa session (`client_min_messages`) à `log`.

Il est possible de recevoir les messages directement dans sa session. Tous les messages de log sont marqués d'un niveau de priorité. Les messages produits par `auto_explain` sont au niveau `log`. Il suffit donc de passer le paramètre `client_min_messages` au niveau `log`.

Positionner le paramètre de session comme ci-dessous, ré-exécuter la requête.

```
SET client_min_messages TO log;  
SELECT...
```

1.11.3 pg_stat_statements

- `pg_stat_statements` nécessite une bibliothèque préchargée. La positionner dans le fichier `postgresql.conf`, redémarrer PostgreSQL et créer l'extension.

Le paramètre à modifier est `shared_preload_libraries` :

```
shared_preload_libraries = 'auto_explain,pg_stat_statements'
```

Si une autre extension (ici `auto_explain`) est également présente, on peut les déclarer ensemble ainsi :

```
shared_preload_libraries = 'auto_explain,pg_stat_statements'
```

Redémarrer PostgreSQL.

Dans la base **postgres** (par exemple), créer l'extension :

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements ;
```

- Inspecter le contenu de l'extension `pg_stat_statements` (`\dx` et `\dx+`).

L'extension installe divers objets, mais seule la vue `pg_stat_statements` nous intéressera ici.

```
\dx+ pg_stat_statements
```

Objets dans l'extension « `pg_stat_statements` »
Description d'objet

```
-----
function pg_stat_statements(boolean)
function pg_stat_statements_info()
function pg_stat_statements_reset(oid,oid,bigint,boolean)
type pg_stat_statements
type pg_stat_statements[]
type pg_stat_statements_info
type pg_stat_statements_info[]
view pg_stat_statements
view pg_stat_statements_info
```

- Vérifier que le serveur est capable d'activer la mesure de la durée des entrées-sorties avec `pg_test_timing`. Puis l'activer (`track_io_timing`), sans oublier de redémarrer PostgreSQL.

`pg_test_timing`²⁴ est livré avec PostgreSQL.

```
# Chemin du packaging RPM du PGDG
/usr/pgsql-17/bin/pg_test_timing
```

(Sur Debian & dérivé, le chemin est `/usr/lib/postgresql/17/bin/pg_test_timing`).

²⁴<https://docs.postgresql.fr/current/pgtesttiming.html>

Testing timing overhead for 3 seconds.
 Per loop time including overhead: 33.24 ns
 Histogram of timing durations:

< us	% of total	count
1	97.25509	87770521
2	2.72390	2458258
4	0.00072	646
8	0.00244	2200
16	0.00984	8882
32	0.00328	2958
64	0.00298	2689
128	0.00099	892
256	0.00055	499
512	0.00016	141
1024	0.00006	53
2048	0.00000	1

`pg_test_timing`²⁵ est livré avec PostgreSQL et mesure les performances de l'horloge système. Si le temps de mesure renvoyé sur la deuxième ligne n'est que de quelques dizaines de nanosecondes, la machine est suffisamment rapide pour que `track_io_timing` renvoie des résultats précis et sans ralentir la requête. C'est le cas sur presque toutes les machines et systèmes d'exploitation actuels, mais il y a parfois des surprises, par exemple dans certaines machines virtuelles ou selon la source de l'horloge système. Sinon, éviter d'activer `track_io_timing` sur un serveur de production. Sur une machine de test ou de formation, ce n'est pas un problème.

Dans le fichier `postgresql.conf`, positionner :

```
track_io_timing = on
```

Changer ce paramètre nécessite aussi de redémarrer PostgreSQL.

- Depuis un autre terminal, créer une base **pgbench** (si pas déjà disponible), l'initialiser (même si elle existait), et lancer une activité dessus en arrière-plan :

```
# en tant qu'utilisateur postgres
createdb -e pgbench
/usr/pgsql-17/bin/pgbench -i -s135 pgbench
/usr/pgsql-17/bin/pgbench -c5 -j1 pgbench -T 600 -P1
```

```
createdb -e pgbench
```

```
SELECT pg_catalog.set_config('search_path', '', false);
CREATE DATABASE pgbench;
```

```
/usr/pgsql-17/bin/pgbench -i -s135 pgbench
```

```
...
creating tables...
generating data (client-side)...
13500000 of 13500000 tuples (100%) done (elapsed 10.99 s, remaining 0.00 s)
vacuuming...
creating primary keys...
done in 14.97 s (drop tables 0.00 s, create tables 0.02 s, client-side generate 11.04
↪ s, vacuum 0.34 s, primary keys 3.57 s).
```

²⁵<https://docs.postgresql.fr/current/pgtesttiming.html>

```
/usr/pgsql-17/bin/pgbench -c5 -j1 pgbench -T 600 -P1
```

```
pgbench (17.4)
starting vacuum...end.
progress: 1.0 s, 2364.9 tps, lat 2.078 ms stddev 1.203, 0 failed
progress: 2.0 s, 2240.0 tps, lat 2.221 ms stddev 0.871, 0 failed
...
```

On a donc 5 clients qui vont mettre à jour la base à raison de 2000 transactions par seconde (valeur très dépendante des CPUs et des disques).

- Dans la vue `pg_stat_statements`, récupérer les 5 requêtes les plus gourmandes en temps cumulé sur l'instance et leur nombre de lignes.

```
SELECT calls, query, rows,
total_exec_time*interval '1ms' AS tps_total
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5
\gx
```

Le résultat va dépendre de l'historique de votre instance, et du temps déroulé depuis le lancement de `pgbench`, mais c'est probablement proche de ceci :

```
-[ RECORD 1 ]-----
calls      | 118591
query      | UPDATE pgbench_accounts SET abalance = abalance + $1 WHERE aid = $2
rows       | 118591
tps_total  | 00:00:22.81672
-[ RECORD 2 ]-----
calls      | 1
query      | copy pgbench_accounts from stdin with (freeze on)
rows       | 13500000
tps_total  | 00:00:08.079233
-[ RECORD 3 ]-----
calls      | 118591
query      | UPDATE pgbench_branches SET bbalance = bbalance + $1 WHERE bid = $2
rows       | 118591
tps_total  | 00:00:04.944426
-[ RECORD 4 ]-----
calls      | 1
query      | alter table pgbench_accounts add primary key (aid)
rows       | 0
tps_total  | 00:00:03.057966
-[ RECORD 5 ]-----
calls      | 118591
query      | UPDATE pgbench_tellers SET tbalance = tbalance + $1 WHERE tid = $2
rows       | 118591
tps_total  | 00:00:01.595914
```

Noter que l'unique ordre `COPY` pour créer la base dure plus que la centaine de milliers d'occurrences de la cinquième requête.

- Quelle est la requête générant le plus d'écritures directes sur disques (*written*) ? Et en temps d'écriture ?

Pour les *written*, il faut tenir compte des trois sources : blocs du cache partagé, blocs des *backends*, fichiers temporaires.

```
SELECT calls,
       pg_size_pretty(8192::numeric
       * (shared_blks_written+local_blks_written+temp_blks_written)) AS written,
       pg_size_pretty(8192::numeric*shared_blks_written) AS shared_written,
       pg_size_pretty(8192::numeric*temp_blks_written) AS temp_written,
       shared_blk_write_time * interval '1ms' AS shared_blk_write_time,
       local_blk_write_time * interval '1ms' AS local_blk_write_time,
       temp_blk_write_time * interval '1ms' AS temp_blk_write_time,
       query
FROM pg_stat_statements
ORDER BY shared_blks_written+local_blks_written+temp_blks_written DESC LIMIT 3
\gx
```

(Cette requête est valable pour PostgreSQL 17; auparavant les champs `shared_blk_write_time` et `local_blk_write_time` étaient regroupés dans un champ `blk_write_time`.)

```
-[ RECORD 1
↪ ]-----+-----
calls                | 1
written              | 3442 MB
shared_written       | 3442 MB
temp_written         | 0 bytes
shared_blk_write_time | 00:00:00.677007
local_blk_write_time  | 00:00:00
temp_blk_write_time   | 00:00:00
query                | copy pgbench_accounts from stdin with (freeze on)
-[ RECORD 2
↪ ]-----+-----
calls                | 374073
written              | 3171 MB
shared_written       | 3171 MB
temp_written         | 0 bytes
shared_blk_write_time | 00:00:07.658022
local_blk_write_time  | 00:00:00
temp_blk_write_time   | 00:00:00
query                | UPDATE pgbench_accounts SET abalance = abalance + $1 WHERE
↪ aid = $2
-[ RECORD 3
↪ ]-----+-----
calls                | 1
written              | 516 MB
shared_written       | 0 bytes
temp_written         | 516 MB
shared_blk_write_time | 00:00:00
local_blk_write_time  | 00:00:00
temp_blk_write_time   | 00:00:00.174825
query                | alter table pgbench_accounts add primary key (aid)
```

Il y a donc beaucoup d'écritures directes (champs `*_written`). C'est le signe que le cache en écriture de PostgreSQL est insuffisant (la base fait 2 Go, à peu près intégralement balayée, et le `shared_buffers` par défaut ne fait que 128 Mo), ou que le `background writer` doit être modifié pour nettoyer plus souvent les blocs *dirty*.

On note que l'`UPDATE` et le `COPY` ont écrit des blocs qui auraient dû passer uniquement par le cache, alors le `ALTER TABLE`, lui, a essentiellement écrit un fichier temporaire (champs `temp_*`), ce qui est logique lors d'une création d'index.

Avec des *shared buffers* plus importants, les `shared_written` sont quasiment absents. Ils proviennent essentiellement d'ordres lourds comme `COPY`.

Quel est le *hit ratio* des requêtes les plus fréquentes?

```
SELECT calls, total_exec_time,
       round(100.0*shared_blks_hit
            /nullif(shared_blks_hit+shared_blks_read, 0),2) AS "hit %",
       query
FROM   pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 5 ;
```

```
-[ RECORD 1 ]---+-----
calls          | 498865
total_exec_time | 96688.18017399928
hit %          | 79.21
query          | UPDATE pgbench_accounts SET abalance = abalance + $1 WHERE aid = $2
-[ RECORD 2 ]---+-----
calls          | 498865
total_exec_time | 24437.844524000397
hit %          | 100.00
query          | UPDATE pgbench_branches SET bbalance = bbalance + $1 WHERE bid = $2
-[ RECORD 3 ]---+-----
calls          | 1
total_exec_time | 8079.232832
hit %          | 100.00
query          | copy pgbench_accounts from stdin with (freeze on)
-[ RECORD 4 ]---+-----
calls          | 498865
total_exec_time | 7014.86179799995
hit %          | 100.00
query          | UPDATE pgbench_tellers SET tbalance = tbalance + $1 WHERE tid = $2
-[ RECORD 5 ]---+-----
calls          | 498865
total_exec_time | 3853.511855000023
hit %          | 100.00
query          | SELECT abalance FROM pgbench_accounts WHERE aid = $1
```

On constate que le *hit ratio* est parfait, sauf la première requête. C'est logique, car la table `pgbench_accounts` ne tient pas dans le cache par défaut et elle est balayée à peu près entièrement par les requêtes de `pgbench`.

1.11.4 PL/Python, import de page web et compression

Sur la base du code suivant en python 3 utilisant un des modules standard (documentation : <https://docs.python.org/3/library/urllib.request.html>), créer une fonction PL/Python récupérant le code HTML d'une page web avec un simple `SELECT pageweb('https://www.postgresql.org/')` :

```
import urllib.request
f = urllib.request.urlopen('https://www.postgresql.org/')
print (f.read().decode('utf-8'))
```

Il faut bien évidemment que PL/Python soit installé. D'abord le paquet, ici sous Rocky Linux 8 avec PostgreSQL 17 (le nom du paquet doit être adapté à la version de PostgreSQL) :

```
# dnf install postgresql17-plpython3
```

Sur Debian, Ubuntu et dérivés ce sera :

```
# apt install postgresql-plpython3-17
```

Puis, dans la base de données concernée :

```
CREATE EXTENSION plpython3u ;
```

La fonction PL/Python est :

```
CREATE OR REPLACE FUNCTION pageweb (url text)
  RETURNS text
AS $$
import urllib.request
f = urllib.request.urlopen(url)
return f.read().decode('utf-8')
$$ LANGUAGE plpython3u COST 10000;
```

Évidemment, il ne s'agit que d'un squelette ne gérant pas les erreurs, les redirections, etc.

Stocker le résultat dans une table.

On vérifie ainsi le bon fonctionnement :

```
CREATE TABLE pagesweb (url text, page text, pagebz2 bytea, page2 text) ;

INSERT INTO pagesweb (url, page)
SELECT 'https://www.postgresql.org/', pageweb('https://www.postgresql.org/') ;
```

Puis stocker cette page en compression maximale dans un champ `bytea`, en passant par une fonction python inspirée du code suivant (documentation : <https://docs.python.org/3/library/bz2.html>) :

```
import bz2
compressed_data = bz2.compress(data, compresslevel=9)

import bz2
c=bz2.compress(data, compresslevel=9)
```

Même si la page récupérée est en texte, la fonction python exige du binaire, donc le champ en entrée sera du `bytea` :

```
-- version pour bytea
CREATE OR REPLACE FUNCTION bz2 (objet bytea)
RETURNS bytea
AS $$
import bz2
return bz2.compress(objet, compresslevel=9)
$$ LANGUAGE plpython3u IMMUTABLE COST 1000000;
```

On peut faire la conversion depuis `text` à l'appel ou modifier la fonction pour qu'elle convertisse d'elle-même. Mais le plus confortable est de créer une fonction SQL de même nom qui se chargera de la conversion. Selon le type en paramètre, l'une ou l'autre fonction sera appelée.

```
-- fonction d'enrobage pour s'épargner une conversion explicite en bytea
CREATE OR REPLACE FUNCTION bz2 (objet text)
RETURNS bytea
AS $$
SELECT bz2(objet::bytea) ;
$$ LANGUAGE sql IMMUTABLE ;
```

Compression de la page :

```
UPDATE pagesweb
SET pagebz2 = bz2 (page) ;
```



NB : PostgreSQL stocke déjà les textes longs sous forme compressée (mécanisme du TOAST).

Tout ceci n'a donc d'intérêt que pour gagner quelques octets supplémentaires, ou si le `.bz2` doit être réutilisé directement. Noter que l'on utilise ici uniquement des fonctionnalités standards de PostgreSQL et python3, sans module extérieur à la fiabilité inconnue.

De plus, les données ne quittent pas le serveur, épargnant du trafic réseau.

Écrire la fonction de décompression avec la fonction python `bz2.decompress`.

```
CREATE OR REPLACE FUNCTION bz2d(objet bytea)
RETURNS bytea
AS $$
import bz2
return bz2.decompress(objet)
$$ LANGUAGE plpython3u IMMUTABLE COST 1000000;
```

Utiliser ensuite `convert_from( bytea, 'UTF8')` pour récupérer un `text`.

```
CREATE OR REPLACE FUNCTION bz2_to_text (objetbz2 bytea)
RETURNS text
AS $$
SELECT convert_from( bz2d(objetbz2), 'UTF8')
$$ LANGUAGE sql IMMUTABLE ;
```

Vérification que l'on obtient au final le même texte qu'avant compression :

```
UPDATE pagesweb
SET page2 = bz2_to_text( pagebz2 )
;
-- Vérification que la page décompressée est identique à l'originale
SELECT count(*) AS pages,
        count(*) FILTER (WHERE page = page2) AS pages_identique
FROM    pagesweb ;
```

pages	pages_identique
1	1

1.11.5 PL/Perl et comparaison de performances

Créer la fonction `multi_replace` en PL/pgSQL à partir du wiki PostgreSQL : https://wiki.postgresql.org/wiki/Multi_Replace_plpgsql

Le code sur le wiki est le suivant :

```
/* This function quotes characters that may be interpreted as special
   in a regular expression.
   It's used by the function below and declared separately for clarity. */
CREATE FUNCTION quote_meta(text) RETURNS text AS $$
SELECT regexp_replace($1, '([\[\]\\\^\$\.\|\?\\*\\+\\(\)])', '\\\\1', 'g');
$$ LANGUAGE SQL strict immutable;

/* Substitute a set of substrings within a larger string.
   When several strings match, the longest wins.
   Similar to php's strtr(string $str, array $replace_pairs).
   Example:
   select multi_replace('foo and bar is not foobar',
       '{"bar":"foo", "foo":"bar", "foobar":"foobar"}'::jsonb);
   => 'bar and foo is not foobar'
*/
CREATE FUNCTION multi_replace(str text, substitutions jsonb)
RETURNS text
AS $$
DECLARE
    rx text;
    s_left text;
    s_tail text;
    res text:='';
BEGIN
    SELECT string_agg(quote_meta(term), '|' )
    FROM jsonb_object_keys(substitutions) AS x(term)
    WHERE term <> ''
    INTO rx;

    IF (COALESCE(rx, '') = '') THEN
        -- the loop on the RE can't work with an empty alternation
        RETURN str;
    END IF;

    rx := concat('^(.*)(', rx, ')(.*)$'); -- match no more than 1 row

    loop
        s_tail := str;
        SELECT
            concat(matches[1], substitutions->>matches[2]),
            matches[3]
        FROM
            regexp_matches(str, rx, 'g') AS matches
        INTO s_left, str;

        exit WHEN s_left IS NULL;
        res := res || s_left;
    END loop;
```

```
res := res || s_tail;
RETURN res;
```

```
END
```

```
$$ LANGUAGE plpgsql strict immutable;
```

Récupérer la fonction en PL/perl sur le même wiki : https://wiki.postgresql.org/wiki/Multi_Replace_Perl.

Évidemment, il faudra l'extension dédiée au langage Perl :

```
# dnf install postgresql14-plperl
```

```
CREATE EXTENSION plperl;
```

Le code de la fonction est :

```
CREATE FUNCTION multi_replace(string text, orig text[], repl text[])
RETURNS text
AS $BODY$
my ($string, $orig, $repl) = @_;
my %subs;

if (@$orig != @$repl) {
    elog(ERROR, "array sizes mismatch");
}
if (ref @$orig[0] eq 'ARRAY' || ref @$repl[0] eq 'ARRAY') {
    elog(ERROR, "array dimensions mismatch");
}

@subs{@$orig} = @$repl;

my $re = join "|", map quotemeta,
    sort { (length($b) <=> length($a)) } keys %subs;
$re = qr($re)/;

$string =~ s/$re/$subs{$1}/g;
return $string;
$BODY$ language plperl strict immutable;
```

Vérifier que les deux fonctions ont le même nom mais des types de paramètres différents.

```
\df multi_replace
```

Schéma	Nom	Liste des fonctions		
		...résultat	Type ... paramètres	Type
public	multi_replace	text	string text, orig text[], repl text[]	func
public	multi_replace	text	str text, substitutions jsonb	func

PostgreSQL sait quelle fonction appeler selon les paramètres fournis.

Le test va consister à transposer tous les noms et lieux des *Misérables* de Victor Hugo dans une version américaine :

Le test va consister à transposer tous les noms et lieux des *Misérables* de Victor Hugo dans une version américaine :

- charger la base du projet Gutenberg si elle n'est pas déjà en place.
- créer une table `miserables` reprenant tous les livres dont le titre commence par « Les misérables ».

```
CREATE TABLE miserables as select * from textes
WHERE livre LIKE 'Les misérables%' ;
```

Cette table fait 68 000 lignes.

Tester le bon fonctionnement avec ces requêtes :

```
SELECT multi_replace (contenu, '{"Valjean":"Valjohn", "Cosette":"Lucy"}'::jsonb)
FROM miserables
WHERE contenu ~ '(Valjean|Cosette)' LIMIT 5 ;

SELECT multi_replace(contenu, '{Valjean,Cosette}', '{Valjohn, Lucy}' )
FROM miserables
WHERE contenu ~ '(Valjean|Cosette)' LIMIT 5 ;
```

Le texte affiché doit comporter « Jean Valjohn » et « Lucy ».

Pour faciliter la modification, prévoir une table pour stocker les critères :

```
CREATE TABLE remplacement (j jsonb, old_t text[], new_t text[]) ;
```

Insérer par exemple les données suivantes :

```
INSERT INTO remplacement (j)
SELECT '{"Valjean":"Valjohn", "Jean Valjean":"John Valjohn",
"Cosette":"Lucy", "Fantine":"Fanny", "Javert":"Green",
"Thénardier":"Thenardy", "Éponine":"Sharon", "Azelma":"Azealia",
"Marius":"Marc", "Gavroche":"Garry", "Enjolras":"Joker",
"Notre-Dame":"Empire State Building", "Victor Hugo":"Victor Hugues",
"Hugo":"Hugues", "Fauchelevent":"Dropwind", "Bouchart":"Butcher",
"Célestine":"Celeste", "Mabeuf":"Myoax", "Leblanc":"White",
"Combeferre":"Combiron", "Magloire":"Glory",
"Gillenormand":"Jillnorthman", "France":"États-Unis",
"Paris":"New York", "Louis Philippe":"Andrew Jackson"}'::jsonb ;
```

Copier le contenu sous forme de tableau de caractères dans les autres champs :

```
UPDATE remplacement
SET old_t = noms_old , new_t = noms_new
FROM ( SELECT array_agg (key) AS noms_old, array_agg (value) AS noms_new
FROM (
```

```
                SELECT (jsonb_each_text (j)).* FROM remplacement
            ) j1
        ) j2 ;
```

On vérifie le contenu :

```
SELECT * FROM remplacement \gx
```

Comparer la performance des deux fonctions suivantes :

```
\pset pager off
-- fonction en PL/perl
EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu, (SELECT j FROM remplacement))
FROM miserables ;

-- fonction en PL/pgSQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu,
                      (SELECT old_t FROM remplacement),
                      (SELECT new_t FROM remplacement) )
FROM miserables ;
```

```
\pset pager off
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu, (SELECT j FROM remplacement))
FROM miserables ;

EXPLAIN (ANALYZE, BUFFERS)
SELECT multi_replace (contenu,
                      (SELECT old_t FROM remplacement),
                      (SELECT new_t FROM remplacement) )
FROM miserables ;
```

Selon les performances de la machine, les résultats peuvent varier, mais la première (en PL/perl) est probablement plus rapide. La fonction en PL/perl montre son intérêt quand il y a beaucoup de substitutions.

1.11.6 hll

- Installer l'extension `hll` dans la base de données de test :
- le paquet est `hll_17` ou `postgresql-17-hll` (adapter le numéro à votre version de PostgreSQL) selon la distribution ;
- l'extension se nomme `hll` ;
- elle nécessite d'être préalablement déclarée dans `shared_preload_libraries`.

Sur Rocky Linux et autres dérivés Red Hat :

```
# dnf install hll_17
```

Sur Debian, Ubuntu et dérivés :

```
# apt install postgresql-17-hll
```

Modifier `postgresql.conf` ainsi afin que la bibliothèque soit préchargée dès le démarrage du serveur :

```
shared_preload_libraries = 'hll'
```

Redémarrer PostgreSQL.

Installer l'extension dans la base :

```
# CREATE EXTENSION hll ;
```

- Créer un jeu de données simulant des voyages en transport en commun, par passager selon la date :

```
CREATE TABLE voyages
(voyage_id      bigint GENERATED ALWAYS AS IDENTITY,
passager_id    text,
d              date
) ;

INSERT INTO voyages (passager_id, d)
SELECT sem+mod(i, sem+1) || '-' || mod(i,77777) AS passager_id, d
FROM generate_series (0,51) sem,
     LATERAL
     (SELECT i,
      '2019-01-01'::date + sem * interval '7 days' + i * interval '2s' AS d
     FROM generate_series (1,
      (case when sem in (31,32,33) then 0 else 22 end +abs(30-sem))*5000 ) i
     ) j
;
```

Cette table de 9 millions de voyages étalés de janvier à décembre 2019 pèse 442 Mo.

- Activer l'affichage du temps (`timing`).
- Désactiver JIT et le parallélisme.

- Passer la mémoire de tri à 1 Go.
- Précharger la table dans le cache de PostgreSQL.

```
\timing on
SET max_parallel_workers_per_gather TO 0 ;
SET jit TO off ;
SET work_mem TO '1GB';
```

```
CREATE EXTENSION pg_prewarm ;
```

```
SELECT pg_prewarm('voyages') ;
```

- Calculer, par mois, le nombre exact de voyages et de passagers **distincts**.
- Dans le plan de la requête, chercher où est perdu le temps.

```
SELECT
    date_trunc('month', d)::date AS mois,
    COUNT(*) AS nb_voyages,
    count(DISTINCT passenger_id) AS nb_d_passagers_mois
FROM voyages
GROUP BY 1 ORDER BY 1 ;
```

mois	nb_voyages	nb_d_passagers_mois
2019-01-01	1139599	573853
2019-02-01	930000	560840
2019-03-01	920401	670993
2019-04-01	793199	613376
2019-05-01	781801	655970
2019-06-01	570000	513439
2019-07-01	576399	518478
2019-08-01	183601	179913
2019-09-01	570000	527994
2019-10-01	779599	639944
2019-11-01	795401	728657
2019-12-01	830000	767419

(12 lignes)

Durée : 57301,383 ms (00:57,301)

Le plan de cette même requête avec `EXPLAIN (ANALYZE, BUFFERS)` est :

QUERY PLAN

```
-----
GroupAggregate  (cost=1235334.73..1324038.21 rows=230 width=20)
    (actual time=11868.776..60192.466 rows=12 loops=1)
    Group Key: ((date_trunc('month'::text, (d)::timestamp with time zone))::date)
    Buffers: shared hit=56497
    -> Sort      (cost=1235334.73..1257509.59 rows=8869946 width=12)
        (actual time=5383.305..5944.522 rows=8870000 loops=1)
        Sort Key:
            ((date_trunc('month'::text, (d)::timestamp with time zone))::date)
        Sort Method: quicksort  Memory: 765307kB
        Buffers: shared hit=56497
        -> Seq Scan on voyages  (cost=0.00..211721.06 rows=8869946 width=12)
            (actual time=0.055..3714.690 rows=8870000 loops=1)
```

Buffers: shared hit=56497
 Planning Time: 0.439 ms
 Execution Time: 60278.583 ms

Le plan est visible sur <https://explain.dalibo.com/plan/Hj> (pour PostgreSQL 14). Il suppose que `shared_buffers` est assez grand pour que tous les accès se fassent en mémoire (*shared hits*). Le `work_mem` élevé permet que le tri des 765 Mo soit aussi en mémoire. Le cas est donc idéal. L'essentiel du temps est perdu en tri.

Pour donner une idée de la lourdeur d'un `COUNT(DISTINCT)` : un décompte non distinct (qui revient à calculer le nombre de voyages) prend sur la même machine 5 secondes, même moins si le parallélisme est utilisé, mais ce qu'un `COUNT(DISTINCT)` ne permet pas.

— Calculer, pour l'année, le nombre exact de voyages et de passagers **distincts**.

```
SELECT COUNT(*) AS nb_voyages,
       COUNT(DISTINCT passager_id) AS nb_d_passagers_annee
FROM voyages;
```

nb_voyages	nb_d_passagers_annee
8870000	4731210

Durée : 60396,816 ms (01:00,397)

On a donc plusieurs millions de voyages chaque mois, répartis sur quelques centaines de milliers de passagers mensuels, qui ne totalisent que 4,7 millions de personnes distinctes. Il y a donc un fort turn-over tout au long de l'année sans que ce soit un renouvellement complet d'un mois sur l'autre.

— Recompter les passagers dans les deux cas en remplaçant le `COUNT(DISTINCT)` par cette expression : `hll_cardinality(hll_add_agg(hll_hash_text(passager_id))):int`

Les ID des passagers sont hachés, agrégés, et le calcul de cardinalité se fait sur l'ensemble complet.

```
SELECT
  date_trunc('month', d)::date AS mois,
  hll_cardinality(hll_add_agg(hll_hash_text(passager_id))):int
  AS nb_d_passagers_mois
FROM voyages
GROUP BY 1 ORDER BY 1 ;
```

mois	nb_d_passagers_mois
2019-01-01	563372
2019-02-01	553182
2019-03-01	683411
2019-04-01	637927
2019-05-01	670292
2019-06-01	505151
2019-07-01	517140
2019-08-01	178431
2019-09-01	527655
2019-10-01	632810

```
2019-11-01 | 708418
2019-12-01 | 766208
(12 lignes)
```

Durée : 4556,646 ms (00:04,557)

L'accélération est foudroyante (facteur 10 ici). Les chiffres sont différents, mais très proches (écart souvent inférieur à 1 %, au maximum 2,8 %).

Le plan indique un parcours de table et un agrégat par hachage :

```
Sort (actual time=5374.025..5374.025 rows=12 loops=1)
  Sort Key: ((date_trunc('month'::text, (d)::timestamp with time zone))::date)
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=56497
-> HashAggregate (actual time=5373.793..5374.009 rows=12 loops=1)
  Group Key: (date_trunc('month'::text, (d)::timestamp with time zone))::date
  Buffers: shared hit=56497
    -> Seq Scan on voyages (actual time=0.020..3633.757 rows=8870000 loops=1)
        Buffers: shared hit=56497
Planning Time: 0.122 ms
Execution Time: 5374.062 ms
```

Pour l'année, on a un résultat similaire :

```
SELECT
  hll_cardinality (hll_add_agg (hll_hash_text (passager_id)))::int
AS nb_d_passagers_annee
FROM voyages;
```

```
nb_d_passagers_annee
-----
4645096
```

(1 ligne)
Durée : 1461,006 ms (00:01,461)

L'écart est de 1,8 % pour une durée réduite d'un facteur 40. Cet écart est-il acceptable pour les besoins applicatifs ? C'est un choix fonctionnel. On peut d'ailleurs agir dessus.

— Réexécuter les requêtes après modification du paramétrage de `hll` :

```
SELECT hll_set_defaults(17, 5, -1, 0);
```

Les défauts sont : `log2m=11, regwidth=5, expthresh=-1, sparseon=1`.

La requête mensuelle dure à peine plus longtemps (environ 6 s sur la machine de test) pour un écart par rapport à la réalité de l'ordre de 0,02 à 0,6 %.

La requête sur l'année dure environ le même temps pour seulement 0,2 % d'erreur cette fois :

```
nb_d_passagers_annee
-----
4741645
```

(1 ligne)
Durée : 1122,149 ms (00:01,122)

Selon les cas et après des tests soigneux, on testera donc l'intérêt de modifier ces paramètres tels que décrits sur le site du projet : <https://github.com/citusdata/postgresql-hll>

- Créer une table d'agrégat par mois avec un champ d'agrégat `hll` et la remplir.

```
CREATE TABLE voyages_mois
(mois          date,
 nb_exact_passagers_mois int,
 passagers_hll hll
) ;

INSERT INTO voyages_mois
SELECT
    date_trunc('month', d)::date,
    COUNT(DISTINCT passager_id),
    hll_add_agg (hll_hash_text (passager_id))
FROM voyages
GROUP BY 1;
```

Cette table d'agrégat n'a que 12 lignes mais contient un champ de type `hll` agrégeant les `passager_id` de ce mois. Sa taille n'est que d'1 Mo :

hll=# \d+

		Liste des relations			
Schéma	Nom	Type	Propriétaire	Taille	...
public	voyages	table	postgres	442 MB	
public	voyages_mois	table	postgres	1072 kB	
public	voyages_voyage_id_seq	séquence	postgres	8192 bytes	

À partir de cette table d'agrégat :

- calculer le nombre moyen mensuel de passagers distincts,
- recalculer le nombre de passagers distincts sur l'année à partir de cette table d'agrégat.

La fonction pour agréger des champs de type `hll` est `hll_union_agg`. La requête est donc :

```
SELECT AVG(nb_exact_passagers_mois)::int AS passagers_mois_moyen,
       hll_cardinality(hll_union_agg(passagers_hll))::int AS nb_passagers_annuels
FROM voyages_mois ;
```

```
passagers_mois_moyen | nb_passagers_annuels
-----+-----
          579240 |          4741645
(1 ligne)
Temps : 17,391 ms
```

L'extension HyperLogLog permet donc d'utiliser des tables d'agrégat pour un `COUNT(DISTINCT)`. De manière presque instantanée, on retrouve la même estimation presque parfaite que ci-dessus. Il aurait été impossible de la recalculer depuis la table d'agrégat (au contraire de la moyenne par mois, ou d'une somme du nombre de voyages).

Avec une fonction de fenêtrage sur `hll_union_agg`, calculer une moyenne glissante sur 3 mois du nombre de passagers distincts.

```
SELECT mois,
       nb_exact_passagers_mois,
       CASE WHEN ROW_NUMBER() OVER() > 2 THEN
           hll_cardinality(hll_union_agg(passagers_hll)
                           OVER (ORDER BY mois ASC ROWS 2 PRECEDING) )::bigint
       ELSE null END AS nb_d_passagers_3_mois_glissants
FROM voyages_mois
;
```

mois	nb_exact_passagers_mois	nb_d_passagers_3_mois_glissants
2019-01-01	573853	
2019-02-01	560840	
2019-03-01	670993	1463112
2019-04-01	613376	1439444
2019-05-01	655970	1485437
2019-06-01	513439	1368534
2019-07-01	518478	1333368
2019-08-01	179913	1018605
2019-09-01	527994	1057278
2019-10-01	639944	1165308
2019-11-01	728657	1579378
2019-12-01	767419	1741934

(12 lignes)

Temps : 78,522 ms

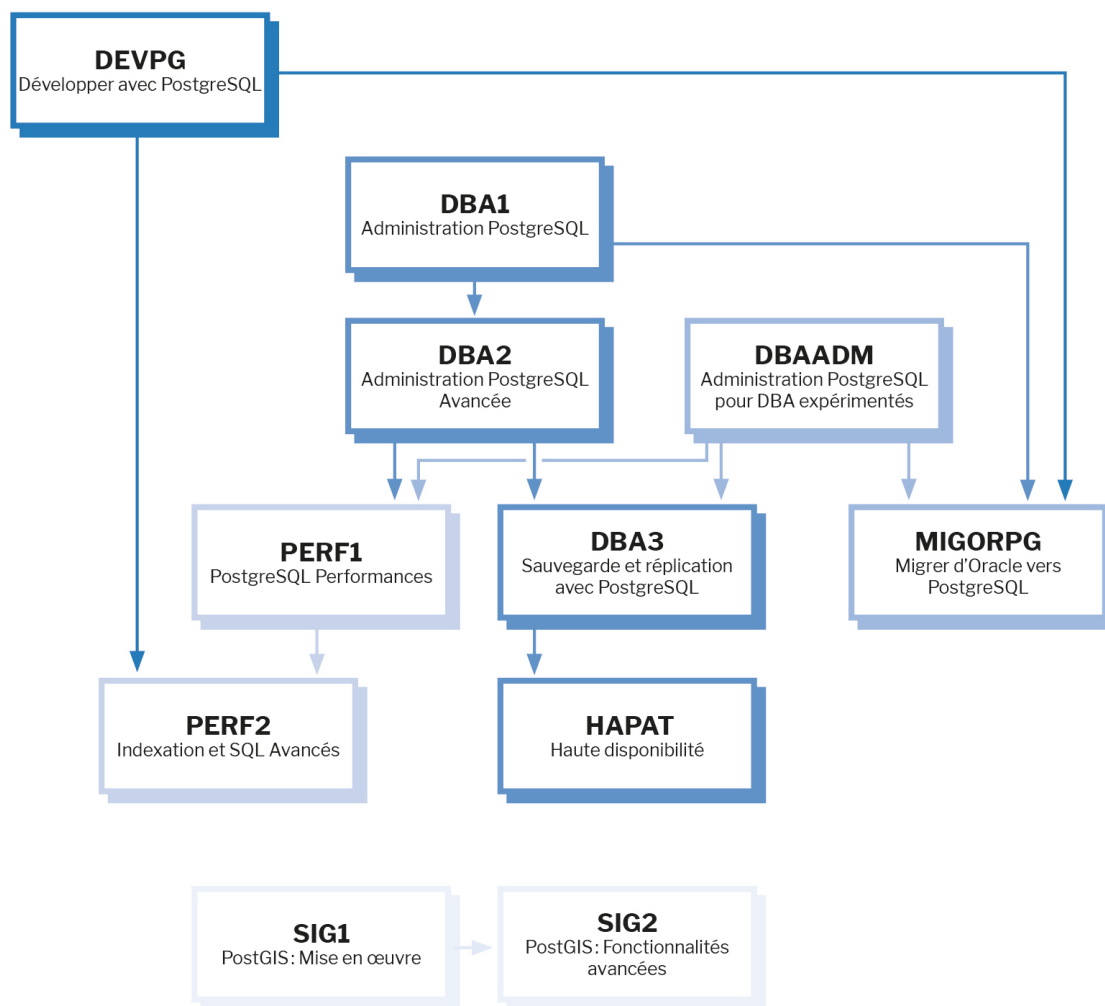
Un `COUNT(DISTINCT)` avec une fonction de fenêtrage n'est en pratique pas faisable, en tout cas pas aussi aisément, et bien plus lentement.

Les formations Dalibo

Retrouvez nos formations et le calendrier sur <https://dali.bo/formation>

Pour toute information ou question, n'hésitez pas à nous écrire sur contact@dalibo.com.

Cursus des formations



Retrouvez nos formations dans leur dernière version :

- DBA1 : Administration PostgreSQL
<https://dali.bo/dba1>
- DBA2 : Administration PostgreSQL avancé
<https://dali.bo/dba2>
- DBA3 : Sauvegarde et réplication avec PostgreSQL
<https://dali.bo/dba3>
- DEV1 : Introduction à SQL
<https://dali.bo/dev1>
- DEVPG : Développer avec PostgreSQL
<https://dali.bo/devpg>
- PERF1 : PostgreSQL Performances
<https://dali.bo/perf1>
- PERF2 : Indexation et SQL avancés
<https://dali.bo/perf2>
- MIGORPG : Migrer d'Oracle à PostgreSQL
<https://dali.bo/migorpg>
- HAPAT : Haute disponibilité avec PostgreSQL
<https://dali.bo/hapat>

Les livres blancs

- Migrer d'Oracle à PostgreSQL
<https://dali.bo/dlb01>
- Industrialiser PostgreSQL
<https://dali.bo/dlb02>
- Bonnes pratiques de modélisation avec PostgreSQL
<https://dali.bo/dlb04>
- Bonnes pratiques de développement avec PostgreSQL
<https://dali.bo/dlb05>

Téléchargement gratuit

Les versions électroniques de nos publications sont disponibles gratuitement sous licence open source ou sous licence Creative Commons.

